H264编码原理-零声学院

H264压缩技术

H.264 两层编码体系

H.264 编解码基本框架

- 编码器框图
- 解码器框图

H.264 的关键技术

- 帧内预测
- 帧间预测
- 变换和量化
- 去块滤波
- 熵编码

H.264有四种画质级别

编码器的算法可定制模块

- （1）视频预处理
- （2）码率控制
- （3）运动估计
- （4）模式选择

X264编码参数

- 编码参数简介
  - （1）视频预处理模块
  - （2）码率控制模块
  - （3）运动估计模块
  - （4）模式选择模块
- 候选测试编码参数

x264 工程分析

编码流程

X264优化

- 参数优化

并行优化

参考

零声学院编辑整理

音视频高级教程 – Darren老师：QQ326873713

课程链接：https://ke.qq.com/course/468797?tuin=137bb271

重点：

为什么存在编码延迟

什么影响编码效率

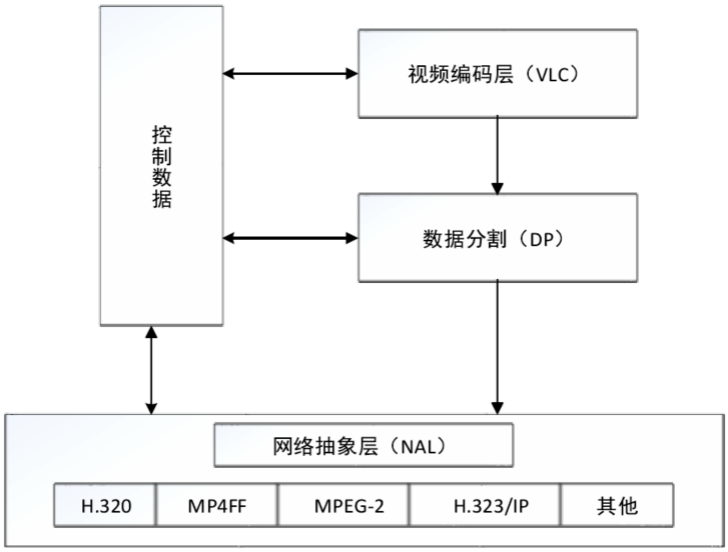
H264压缩技术

- 帧内预测压缩，解决的是空域数据冗余问题。什么是空域数据，就是这幅图里数据在宽高空间内包含了很多颜色，光亮。人的肉眼很难察觉的数据。 对于这些数据，我们可以认作冗余。直接压缩掉的。
- 帧间预测压缩，解决的是时域数据冗余问题。在我们之前举例说明过，摄像头在一段时间内所捕捉的数据没有较大的变化，我们针对这一时间内的相同的数据压缩掉。这叫时域数据压缩。
- 整数离散余弦变换(DCT)，将空间上的相关性变为频域上无关的数据然后进行量化。这个比较抽象。这个跟数学是紧密联系在一起的。如果对傅里叶变换理解的比较好的。对这个会理解的比较快。如果对傅里叶变换不了解的。可能有稍许困难。傅里叶变换可以把一个复杂波形图变换成许多的正弦波。只是他们之间的频率不一样。以及振幅也不一样。如果它们在频率上没有一致性那么我们就可以对他进行压缩处理。
- CABAC压缩：无损压缩。

H.264 两层编码体系

H.264 的码流结构，相比于之前的视频标准，虽然都采用了基于块匹配的混合编 码方式，但在层次划分上有着较大的改进。从 H.261 到 H.263 视频编码标准中，将码流的结构划分为四个层次[19]，按照从上到下的顺序依次为：图像层（picture layer）、块组层（GOB layer）、宏块层（macroblock layer）和块层（block layer）。从层次的划

分上可以看出没有针对网络传输部分进行层级的定义和专门设定，H.264 则针对视频数据经由网络传输的需求，从功能上划分为两层，分别是视频编码层（VCL）以及网络抽象层（NAL），如图 2.1 所示。



VCL 层即是视频编码层[20]，进行视频数据的压缩和编码，它包含着以块为基本单位的运动补偿等混合编码技术。经过编码后的数据按照 NAL 规定的格式封装成
NAL 单元，实现了在网络中传输的数据格式的统一，方便和网络协议进行对接。NAL
单元由两部分组成，NAL 头部和包含着编码后视频数据的载荷（RBSP：Raw Byte
Sequence Payload）[21]。NAL 使用下层网络的分段方式来对数据进行封装，其中包含
如何组成一帧数据、信道的信令和如何利用定时信息时间戳等。

H.264 编解码基本框架

作为视频编解码标准 H.264，它对编码后的视频比特流的句法和如何对比特流进行解释作出了规定，但并没有具体地规定如何去实现一个编解码器。在工程中使用的
编解码器建立在相同的语法和框架之下，但对于如何实现并没有做具体的要求，这样
给工程实现提供着更大的发展空间，有利于针对不同硬件设备进行具体实现，有着较
大的灵活性

编码器框图

H.264 的编码器框图如图 2.2 所示，可以看出由图中实现和虚线分为两部分，前向部分和后向部分，后向表示的是重建分支。

- 编码器（前向路径）：编码器在输入的视频帧中以 16x16 的像素宏块为单位做帧内或者帧间处理，随后生成预测宏块 P，对原始块和预测宏块做残差，随后对残差数据进行变换和量化等操作。对于帧内编码，其预测值通过正在被编码的当前帧先前被编码数据，在进行重建宏块像素后求得预测宏块 P；对于帧间编码由对参考帧进行运动估计后求得运动矢量，并在进行运动补偿后求得预测值。

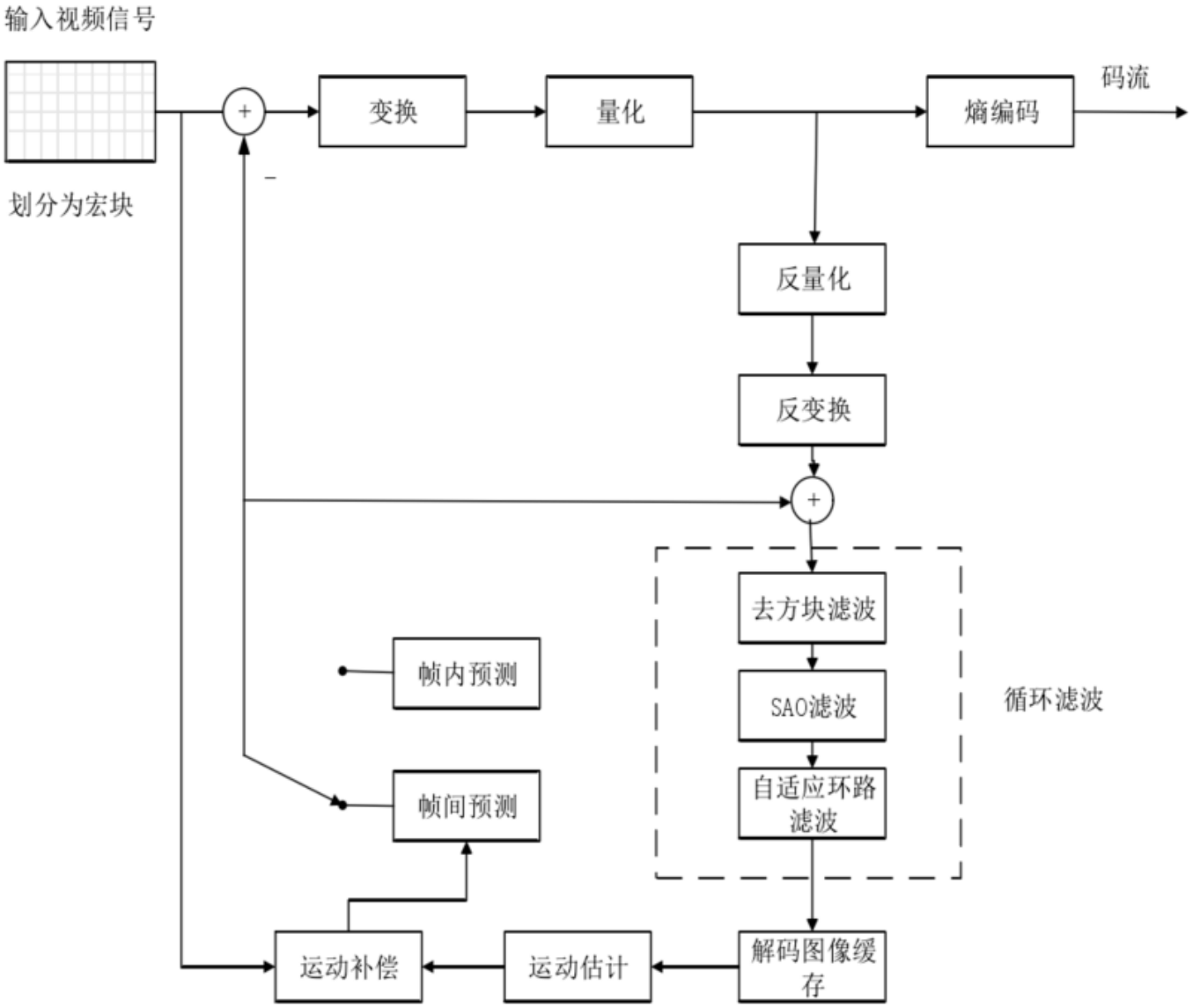


图 2.2 H.264 编码框架图

- 编码器（重建路径）：解码器相对于编码器而言，对视频数据进行解码操作和 反量化、反变换等等。在解码过程中通过一系列操作重建图像后用于对下一帧图像进行预测。经过变换和量化后的宏块得出数据进行反量化和方变换等操作得出差分宏块，由于在反量化和反变换过程为了方便计算，结合不同的量化步长对计算系数进行处理，所以得到的重建后宏块和原始块并不是完全相同。经过对预测宏块与残差宏块相加等操作后得出重建参考帧。

关键技术点：

- 帧内预测压缩，解决的是空域数据冗余问题。
- 帧间预测压缩（运动估计与补偿），解决的是时域数据冗徐问题。
- 整数离散余弦变换（DCT），将空间上的相关性变为频域上无关的数据然后进行量化。
- CABAC压缩。

解码器框图

H.264 的解码器框图如图 2.3 所示。由于 H.264 中加入了视频流在网络传输的支持，解码器从后侧输入的数据包格式为 NAL，码流方向从右至左。解码器在网络抽象层 NAL 中获取到压缩后的码流，然后从码流中提取一帧数据，经过熵编码、重新排序、反量化和反变换后作为加法器的输入，其中包含着重建图像所需要的残差数据和运动矢量等进行解码操作所必需的信息。接下来，解码器通过在码流中解码所获得的头部信息创建出一个预测块 P，将之前求得的残差信息 D 和预测块 P 求和，求得图像块数据 uF’n，然后经过去块滤波，SAO 等操作获得重建图像的解码块 F。

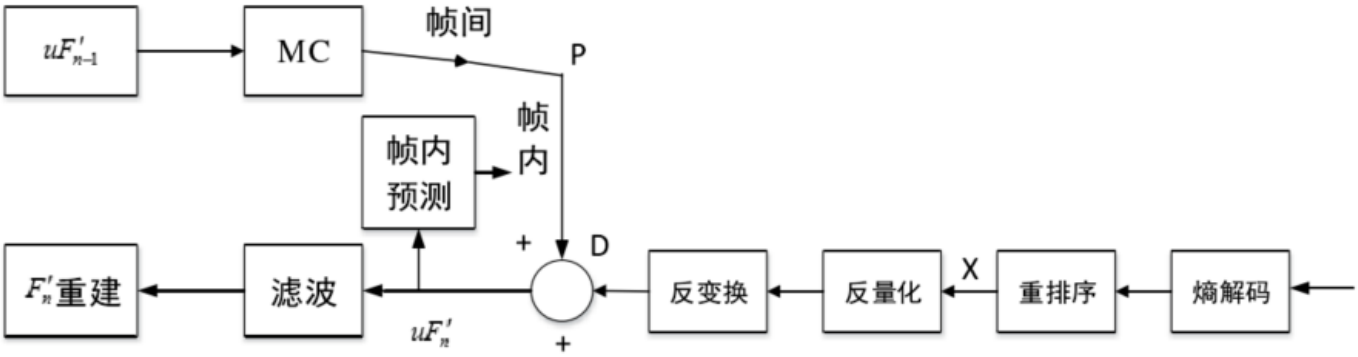


图　2.3 H.264 解码框架图

H.264 的关键技术

H.264 相比于之前的视频编码标准增加了较多的新技术，以适应视频发展的需求，提供了一种更加高效的视频编码标准，并有着较好的网络适用性。标准中使用到的主要技术包括：运动估计、运动补偿、变换、量化、滤波、SAO 和熵编码。视频信息中包含最多的冗余信息是空间冗余信息和时间冗余信息，编码中采用帧内编码和帧间编码的方式较大程度的减少了冗余信息。算法复杂度更为复杂，在计算方式上更加多样，可以更好的针对不同图像的特点进行分析使用，为视频压缩提供更好的压缩性能和方案选择，同时提升图像压缩质量。但是大量复杂的算法也增加了编码需要的时间，从而加大了软件和硬件的实现难度。下面对 H.264 的关键技术进行介绍。

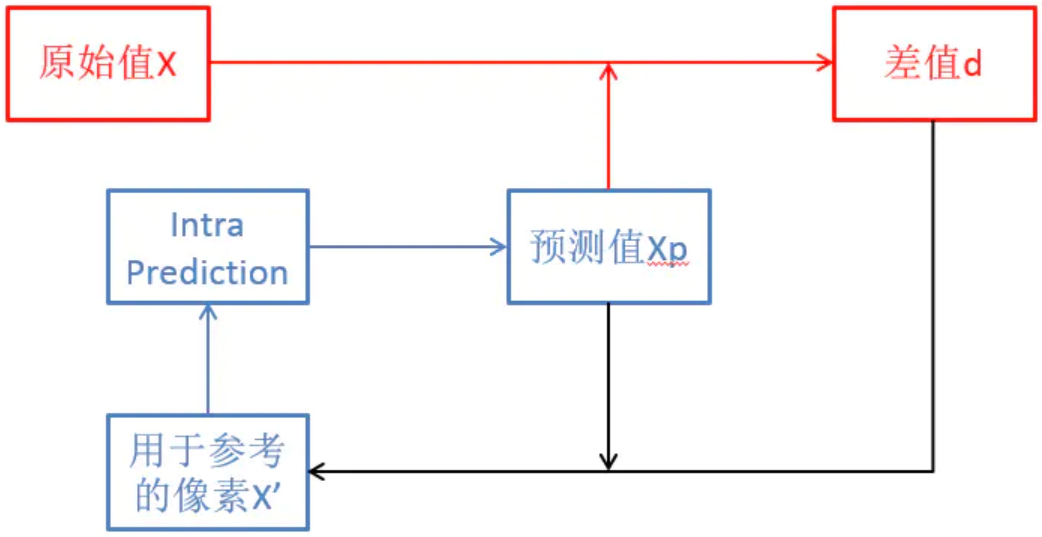
帧内预测

为什么要有帧内预测？因为一般来说，对于一幅图像，相邻的两个像素的亮度和色度值之间经常是比较接近的，也就是颜色是逐渐变化的，不会一下子突变成完全不一样的颜色。而进行视频编码，目的就是利用这个相关性，来进行压缩。

很好理解，存储一个像素的亮度值可能需要8个bit，但是如果相邻的两个像素变化不大，我存储一个像素的原始值，以及第二个像素相对第一个像素的变化值，那么第二个值我可能用2个bit就够了，这就节

约了很多的空间。而节约存储消耗的bit数，也就是节约码率，贯穿了H.264编码器的所有过程，不管是帧内预测、帧间预测、变换、量化、熵编码，一切的一切都是为这个目的服务的，明白了这一点，我们就能轻易的理解H.264编码器，所有看上去复杂难懂的地方，都是为了一个目的——节约码率。

说回到帧内预测，帧内预测的流程见下图



首先，是上图中蓝色的部分，假设现在我需要对一个像素X进行编码，在编码这个像素之前，先假设我已经有一个参考像素X'了，这个参考像素与同一帧的临近像素有关，根据参考像素X'的值，我得到了一个预测值Xp。

然后，是上图中红色的部分，我用编码的像素X减去预测值Xp，得到了残差d，这个残差d代替原始值X被编码进最终的图像，起到了节省码率的作用

最后，是上图中黑色的部分，残差d和预测值Xp相加，得到了X'，用于下一个像素的预测。

总结起来就是三步：

1. 以同一帧图像内的临近像素作为参考，计算预测值Xp
2. 原始值X和预测值Xp的差值d，被传递到解码端
3. 解码端接收到差值d，将其与预测值Xp相加，就得到了“原始值”X'，X'=Xp+d

   传输的是Xp值和d。

步骤很简单，但是里面有几个问题要明确，首先要知道的是，我们固然可以按像素来进行预测，但是这样太费事了，要计算很多次，而且由于帧内预测的特性，你要预测当前的像素，能参考的像素只能是它的临近像素，也就是前面已经编码完成的，它后面的像素还没有编到，所以自然是不能用来编码当前像素的，因此我们只能一个像素一个像素进行编码，编完了一个才能编下一个，这样显然会很慢。

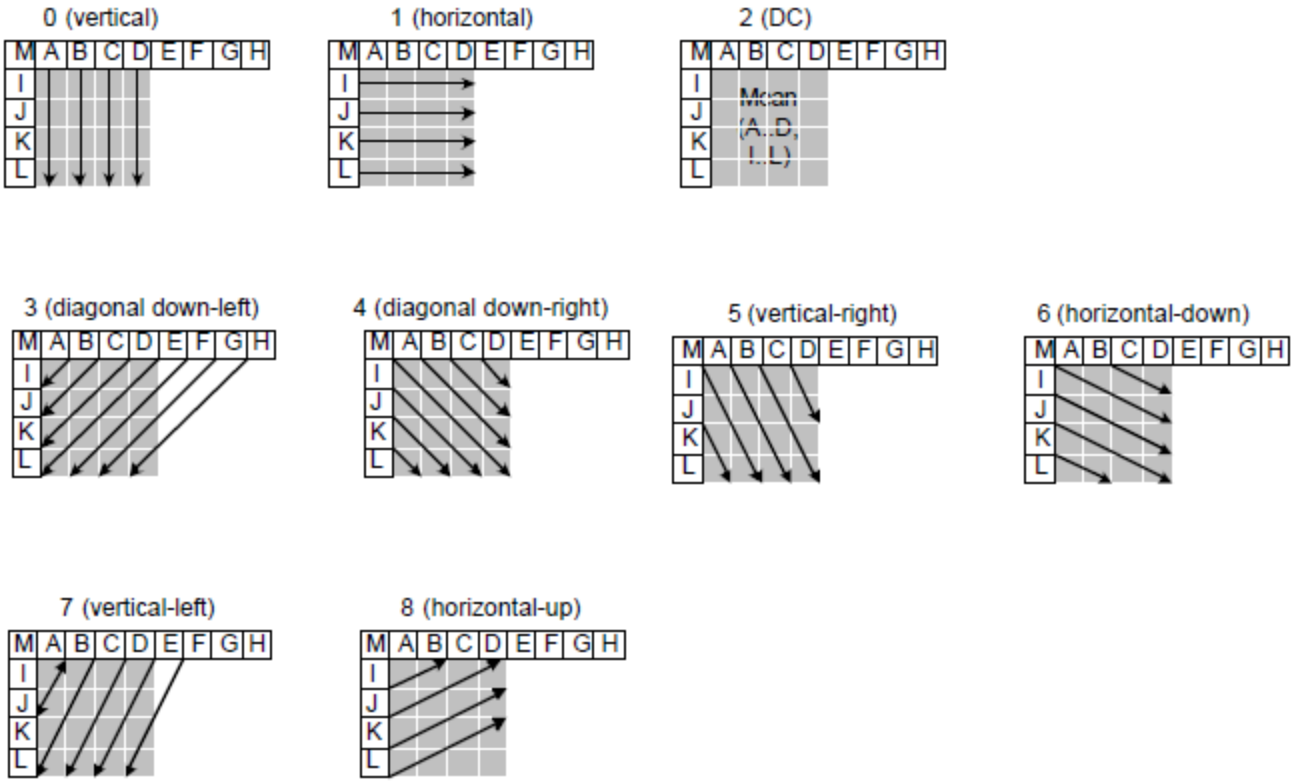
于是H.264标准中提出**按块进行计算**，一个宏块是16x16像素，然后它可以分成子块，最小是4x4的（这个大小是对于亮度编码而言，至于色度编码，4:2:0格式的色度宏块的长和宽都是亮度宏块的一半），这样也

能大大提高计算速度。因此下面提到的“值”可以代表“像素”也可以代表“块”，从原理上来说是一样的，而实际采用的是“块”，因此我也就统一用块这个词了。

我们从上面三个步骤从头开始一点点找问题

1. 这个参考值X'是怎么来的？答案是在第三步里，使用Xp和传递过来的残差d相加得到的，这个X'用于后面的块编码时的参考。
2. 预测值Xp是怎么从X'获得的？答案是根据X'，通过某个公式计算得到的。而这个X'并不是只有一个块，而是有左侧、左上、正上、右上一共四个块作为参考。
3. 上面提到的这个某个公式是什么？根据白皮书，Intra（帧内预测）有两种，一种是4x4大小的亮度块，一种是16x16大小的亮度块。

对于4x4大小的亮度块，我们有9种预测模式，如下图所示



- 对于预测模式0（vertical），当前块的十六个像素值，完全由其上方块最后一行的那四个像素值决定，第一列所有的Xp值都等于A，第二排都等于B，以此类推。
- 预测模式1（horizontal）也是一样，完全由图中IJKL四个像素值决定。
- 对于预测模式2（DC），则十六个像素值完全相等，等于ABCDIJKL这八个像素的平均值。
- 对于预测模式3-8，倾斜方向的，各个像素是由A到L像素通过权重不等的公式加权计算的

M	A	B	C	D	E	F	G	H
I	a	b	c	d				
J	e	f	g	h				
K	i	j	k	l				
L	m	n	o	p				

Figure 2 Labelling of prediction samples (4x4)

比如对于模式3（diagnal down-left）来说，a=(A+2B+C+2)/4，这里+2代表四舍五入，b和e=(B+2C+D)/4，cfi=(C+2D+E+2)/4，dgjm=(D+2E+F+2)/4，hkn=(E+2F+G+2)/4，lo=(F+2G+H+2)/4，p=(G+2H+H+2)/4=(G+3H+2)/4

对于模式4（diag down-right），加权系数也是(1,2,1)/4，afkp用IMA三个像素计算，以此类推

对于模式5（vertical right），aj=(M+A+1)/2，同理bk是AB均值，cl是BC均值，d是CD均值（因为这几个像素延长线都不在预测用的13个像素里面）。en在M的延长线上，所以等于(I+2M+A+2)/4，同理fo\gp\h\i\m都可以计算出来

同理模式6、7、8也基本跟5一样，注意对于模式8来说，klmnop四个像素都等于L，因为其延长线在L的下面和前面，没有可以平均的像素，于是只好用L一个值代替了。

还有一点要注意的是，这里面A到L的像素有的没有怎么办？对于模式2（DC），有什么用什么，反正是求均值就对了。其余的模式就必须用到的那几个像素都存在才行（EFGH四个像素可以不存在，此时认为都等于D）

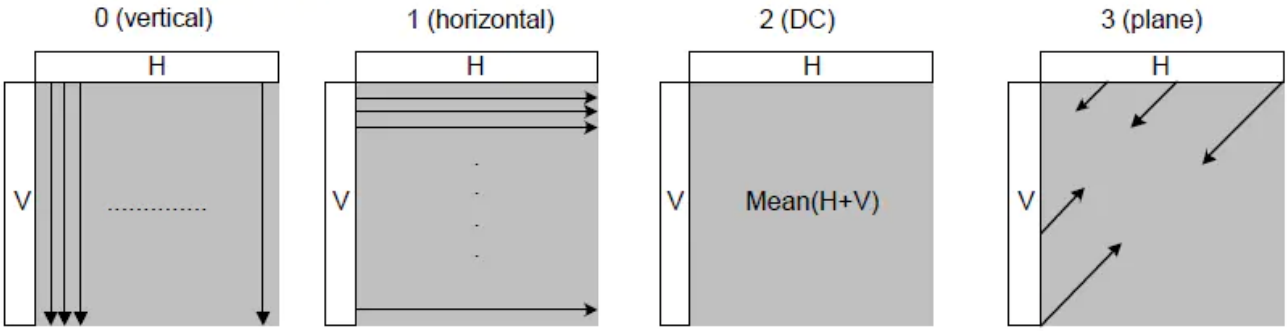
最终，我们要选择哪种模式进行预测？为了在后面节约码率考虑，当然是预测的越准越好，也就是选择Xp和X'差距最小为好。而评判这个差距其实也有好几种算法，比如SAD、SATD等，等到了变换那里再详细说。总之我们用一个公式把上面9种模式的预测都评价了一番，选出里面最好的一种，作为4x4帧内预测的选择。

即是有损在于预测模型的误差。

我们注意到这里有9种模式，之后要进行编码的话，我们除了把残差编进去，总得知道我预测的时候用了哪种模式吧，9这个数就尴尬了，因为刚好三个比特可以表示8种，四个比特可以表示16种，所以3bit不够

4bit又浪费了。怎么办呢？有个方法就比较巧妙，我有1bit用来表示我当前用的模式和前面的是不是一样的，因为经常有这样的情况，我前面块用的预测方向和现在这个块用的预测方向一样（比如物体边缘是一条直线，那么对应的那几个块用的预测方向很可能都是一样的），如果一样，我只用1bit就足够存储了，如果不一样，我再用用4个bit存储，也就达到了节约bit的目的。

说完了4x4的亮度块，我们看看16x16大小的亮度块。16x16亮度块有四种模式，如下图



H.26L Intra 16x16 prediction modes (all predicted from pixels H and V)

Figure 5 Intra 16x16 prediction modes

前面三种很好理解了，重点是第四种plane的计算方式，从图上看大概能理解，不过具体算法我还不是很理解。

我们假设左上角起，上方那一行是17个像素是a1 b2 c3 d4 e5 f6 g7 h8 i9 j8 k7 l6 m5 n4 o3 p2 q1，用这17个像素计算一个H值。

我在上面标了1~9~1的数字，有数字相同的8对像素，后面计算的时候，都是一对对的计算的。

i9;j8 – h8;k7 – g7;l6 – f6;m5 – e5;n4 – d4;o3 – c3;p2 – b2;q1 – a1，这九对分别乘以权重0到8（也就是i9这个像素没有用到），而最左边和最右边两个像素权重最大。

同理V值，也是一样的算法，从上到下像素记作带'的（当然a1' = a1）。

然后计算一个A值，它等于右上角（q1）和左下角（V值计算时候对应的那个q1'）的和乘以16

计算一个B值，它等于（5*H+32）/64，计算一个C值，C=(5*V+32)/64，这种后面加了32又除以64的，其实都是用来四舍五入的。

然后就能计算我们的预测值了。

i00 = A – 7*B –7*C + 16

pix[0][0] = i00 / 32（超出0到255范围的要截断成0或者255）

然后计算第一行十六个像素分别是i00 + B到i00 + 15*B，然后除以32（然后截断到0~255）

第二行是在第一行基础上加了一个C，一直到第16行，加了15个C，于是这256个像素都算出来了。
这就是plane方式的算法。算出来结果就跟下图一样。

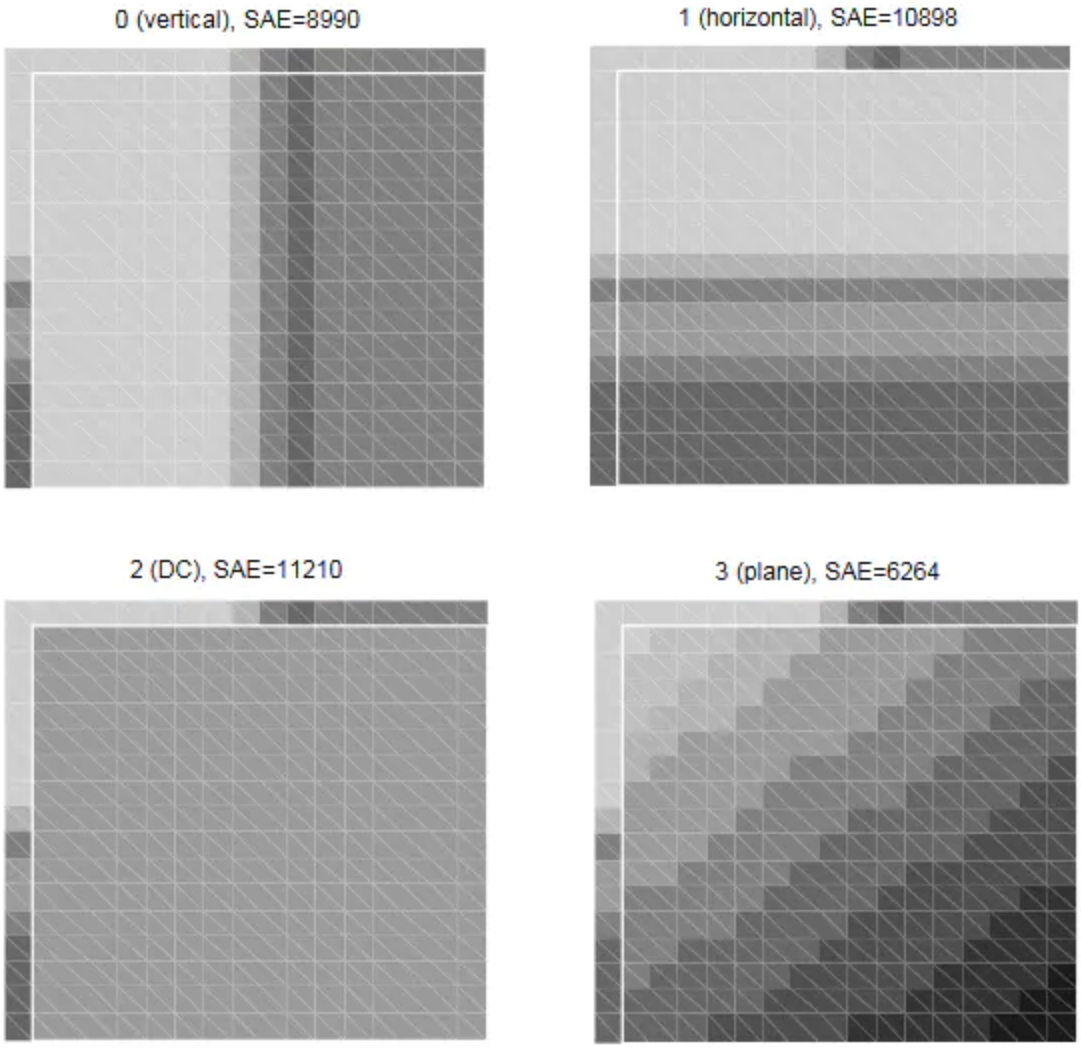


Figure 7 Intra 16x16 predictions

对于色度块是亮度块的四分之一，也就是8x8的，那就只有一种了，预测模式也跟亮度16x16块的类似，有四种，只不过具体的序号不一样而已。是0代表DC，1代表horizontal，2代表vertical，3代表plane。
plane的算法和上面16x16的类似，只不过系数变了，而且两个色度块用的方式一定都是一样的。
好了，我们回到前面的帧内预测的步骤，还有几个问题没有解决

4、重建值X'和原始值是不是一样的？答案是不一样，因为d在传到解码端的过程中经过了量化、变换、反变换和反量化，有了精度的损失，因此Xp+d得到的X'跟原始的X是不一样的。这也是为什么要用重建值X'来得到Xp而不是用原始值X。因为解码器那边在解码图像的时候，用到的是有损的X'获得Xp，如果编码

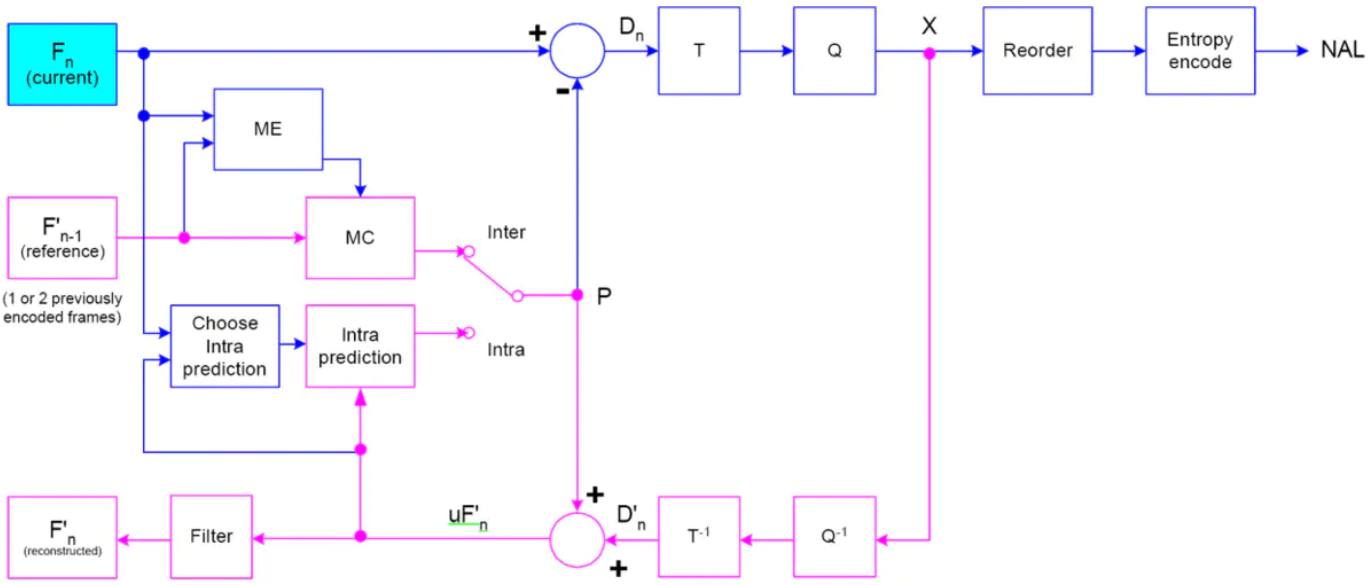
器用无损的X来得到Xp的话，那么得到的残差d在编码侧和解码侧就不一致了，这个误差扩散到了下一个块里。

好了，这样上面的步骤就组成了一个循环，可以一直编码下去了，当然最开始没有参考的那个块，其预测值Xp又是怎么来的呢？可以有一种方式叫做DC128，也就是认为这个块的每个像素都是0x80，然后依据此来计算。

实际上对于一个16x16的宏块，上面这个16x16的四种模式和它16个子块（大小4x4）的9种模式都会算一遍，然后用16个子块的SATD最小值和16x16的四种模式的SATD最小值比较，选择更小的那个，所以这里面计算量还是很大的。x264源代码里有很多节约计算量的设计，不过这些就得看代码了，白皮书里面是没有的，以后再做研究。

另外x264里面还有Intra8x8亮度块的模式，这种我认为应该也是为了找到一个预测最准的模式而加进去的，而且也不常用，所以也就不多介绍了。

回过头来看最开始的那个步骤图，跟白皮书里给的编码器流程图，是不是正是编码器的一部分呢？



进行帧内预测，首先确定预测块和当前块，对于当前块而言，和它而言相邻的宏块作为预测块。将帧内预测的计算放在变换编码之前，可以提升帧内编码的效率。对于不同的图像信息，有着不同的特点：

- 图像中有些区域较为平坦，这类区域附近的颜色变化很小，像素值变化幅度很低，可以用相邻像素块的值来表示当前像素块的值；
- 有些区域在一定方向上变化较小，就可以利用这一特点，使用同一方向上的像素块进行预测。

像素点由亮度信号和色度信息表示：

- 对于亮度信号，将块的大小划分为 4x4和 16x16 两种形式，以此为基本单位进行帧内预测。不同的块划分形式对应着不同的预测模式，有 9 种预测模式对应于 4x4 方式，有 4 种可以选择的方式对应于16x16方式。
- 对于色度信号而言，划分为 8x8 块，有 4 中模式可以选择。

以上是对图像数据进行预测操作然后输出，I_PCM 模式能够使帧内预测模块不对图像数据进行预测操作而直接输出，比如当图像信息杂乱无章，相关性很小的情形下，不进行帧内预测操作会比使用不同的预测方式更加简便而且有效，同时在此模式下对图像的质量不造成影响。

帧内预测操作主要是：首先使用一些预测方式，对图像进行预测，接着进行判断，选择出最优的预测方式，最后根据确定出的预测方式对宏块进行预测。

1. 亮度块预测 4x4

对于 4x4 亮度块，有沿着不同方向的 8 种预测方式以及 DC 帧内预测方式。如图 2.4 中表示了 8 个不同的方向，箭头的指向代表着 8 种预测方向。可以看出都是沿着不同的方向通过相邻块进行预测。在 H.264 中通过语法元素的值来表示采用哪种帧内预测方式，以便于解码端使用。

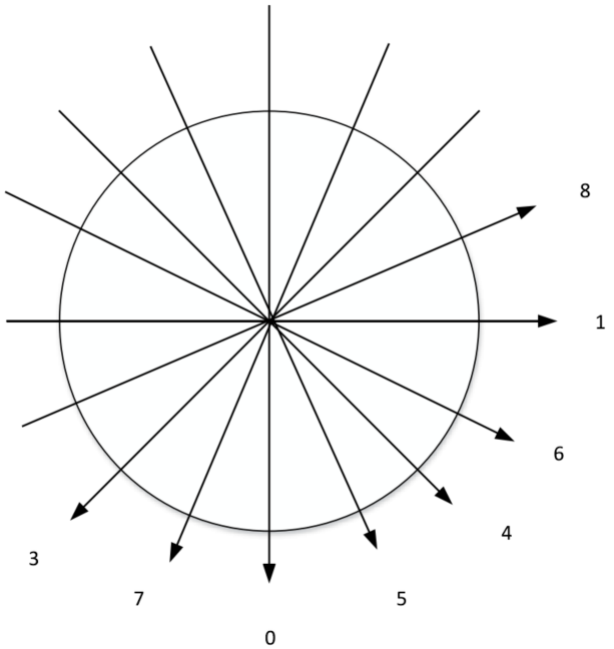


图 2.4 4x4 亮度块预测方向

由图 2.5 中可以看出每一像素点都可以使用与当前像素块临近的 17 的像素进行 计算，如对每个参与运算的像素点使用不同的权重，然后像素点值相加。其中 a、b至 p 表示等待被预测的像素点，A、B 至 M 表示已经经过编码的像素点，选择一种方向进行预测，就意味着使用不同方向的像素点并使用相应的权

值进行计算，例如 m的值可以通过（J+ZK+L+2）/4 来进行预测，也可以使用 DC 预测模式，经由（A+B+C+D+I+J+K+L）/8 进行预测。

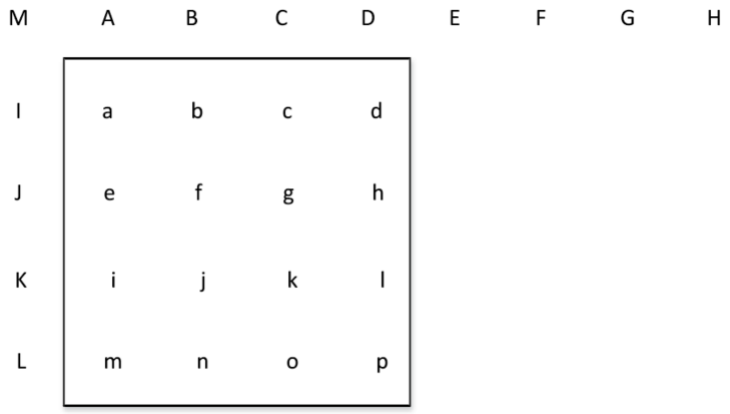


图 2.5 4x4 块相邻像素

2. 亮度块预测

16x16 当对 16x16 亮度块进行预测时，有 4 中不同的预测方向可以选择，分别是垂直方向、水平方向、平面方向和直流方向。4x4 亮度块附近有 17 个像素点，16x16 亮度块附近有 33 个像素点，可以用于预测计算。为了标志 33 个点，对其划分坐标系，横坐标为–1 时，纵坐标从–1 到 15 共 17 个点，当纵坐标为–1 时，横坐标范围从 0 到 15 共 16 个点。使用这 33 个像素点的前提是这些相邻的像素点都可用，如果被预测的宏块在图像的边缘区域或者相邻的宏块和当前宏块处在不同的 slice 中时，则不能使用。

帧间预测

视频帧分为 I、P 和 B 帧三种帧类型[23]，I 帧为信息帧，只使用帧内预测，解码 时不依赖与其他帧，P 帧为单向预测帧，预测时需要使用 I 帧或者 P 帧信息，B 帧为双向预测帧，可以使用 I 帧、P 帧和 B 帧进行预测，表示一帧图像所用的数据量最小，压缩率最高，但是解码时运算量较大。通常来说，I 帧的压缩比为 7，与 JPG 格式的
图片压缩率基本一致，P 帧压缩率为 20 左右，而 B 帧可以达到 50。由此可以看出帧间预测可以很大程度的对视频文件进行压缩。

由于运动是连续的，所以在时间上连续的相邻帧中的大部分元素是相同的，具有时间相关性，利用帧与帧之间的时间相关性就可以对图像进行压缩。物体运动时，通常是由图像中的一个位置移动到另外一个位置，运动的主体不变，在移动范围较小的前提下，运动背景也具有高度相似性，利用这一特性，选定参考帧后，设定坐标系，比较参考帧和被预测帧，通过运动物体在参考帧中的坐标和被预测帧中的坐标计算出运动矢量，使用运动矢量结合参考帧中的信息就可以表示出当前图像。

在帧内预测时，将一副图像划分成几种类型的预测块，对于 16x16 的分块，可以 划分为 1 个 16x16 的块，或者 2 个 16x8 块，或者 4 个 8x8 块；而对于 8x8 的分块可以划分为 2 个 8x4 块或者、2 个 4x8 块或者 4 个 4x4 块。H.264 中可以划分出多种分块模式，其中最小的分块为 4x4，更加准确的表达出小于 4x4 块的物体的运动情况的
同时（即计算出的运动矢量更加精确），减少了编码数据量。

帧间预测中，按照分块标准将一帧图像划分成不同大小的分块，通过参考帧中大小相同的宏块进行预测得出编码的宏块，其中计算产生运动矢量即两个宏块之间的偏移距离。然后使用参考宏块和预测块计算出残差数据，选用更小的像素为基准进行运动估计，求得的运动矢量更加准确，运动矢量精度的提高能够减少残差，如此能够使得在提高了压缩率的同时，更好的保证了视频质量。H.264 中通过使用块匹配算法来进行运动矢量的估计，通常对块以整像素为单位进行运动估计，也可以以亮度分量的1/4 像素为单位或者色度分量的 1/8 像素为单位进行运动估计。这些非整像素是通过插值计算的方式求得的，对图像进行了细化。在选择匹配块时，需要计算参考宏块和预测块之间的相似度，判断其相似度使用归一化的二维相关函数，两个块之间亮度像素点的均方差 MSE，或者计算两个块之间的绝对值的平均值 MAD 等。由于 MAD 的计算过程只有加减和求绝对值运算，没有乘法和除法操作，计算用时少，方便简单，
因此多使用 MAD 的计算方式。其公式如 2-1 所示：

$$SAD(i,j)=\sum_{m=0}^{M}\sum_{n=0}^{N}\left|f_k(m,n)-f_{k-1}(m+i,n+j)\right| \qquad (2\text{-}1)$$

帧间预测首先计算运动矢量和参考索引值。在 skip 模式下参考索引的值为 0， 而其他模式下编码模式所使用的参考索引都是通过计算由编码码流获得。运动矢量的
运动搜索算法很多，主要有全局搜索算法（Full Search Method，FS）、三步法（Three Step Search，TSS），四步法（Four Step Search，FSS）、二维对数法（Two Dimensional Logarithmic，TDL）、交叉法（Cross Serach Algorithm,CSA）、新三步法（New Three Step Search，NTSS）、新四步法（New Four Step Search,NFSS）和菱形法（Diamond Search，DS）等。

全搜索算法[24]是对采用此种预测算法的块，在预测帧中使用 SAD 对帧内所有预测块进行计算，求出 SAD 最小的块，作为算法作用块的预测块，随后根据选定的不同像素标准，如全像素或者亚像素，求出两个块之间的坐标偏移量即运动矢量，再求得两个块之间的残差。全搜索算法的优点是可以从预测帧中选择出最准确的预测块，为全局最优结果，预测精度很高。但是这种算法每确定一个块的预测块都需要对预测帧中的所有块做运算，算法复杂度高，搜索范围太广，大大增加了视频压缩时间。

三步搜索算法[25]是快速搜索算法中较为常用的算法。它的计算规模相比于全搜索算法，是后者的 1/10 左右。此算法的思想如下：首先选定开始进行搜索的中心坐标，然后确定进行搜索的范围，搜索步长初始值设定为最大搜索范围的一半。在以起始点 为中心的正方形上对待预测块计算匹配误差，如果在中心点处，计算的 SAD 值最小，则认为待预测块没有运动，预测块所处的位置即为中心点；若在正方形上的其余 8个点上计算的 SAD 最小，则以最小点所处位置为中心重新确定正方形，此时搜索步长为上一次的二分之一，将第三次搜索得出的结果作为最终结果。此种搜索算法的计算复杂度较小，但是容易由于计算出的点是局部最优，而不是全局最优导致匹配误差较大。

菱形搜索算法[26]为使用度最高的搜索算法，可以选用大菱形搜索算法（LDSP）或者小菱形搜索算法（SDSP）如图 2.6 所示：

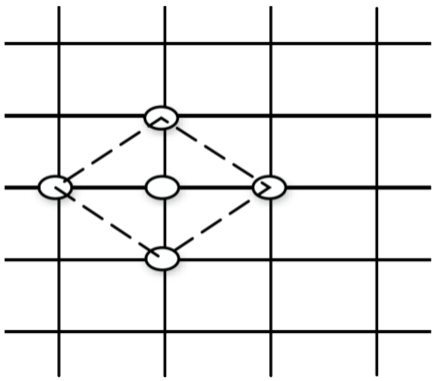


图 2.6 菱形搜索示意图

大菱形搜索算法中采用 9 个点进行搜索，中心点和按照菱形顶点确定的 8 个像素 点，小菱形搜索算法中使用 5 个点进行搜索，中心点以及垂直和水平方向上的 4 个像素点。具体算法思想是，首先和三步搜索算法相同，确定搜索的中心点和搜索步长，然后加入以此为中心的菱形 8 个顶点（大菱形搜索算法）或者 5 个顶点（小菱形搜索
算法），对这些点计算 SAD 值，从中得出 SAD 值最小的点，如果最小值的点为中心点则判定没有运动，算法结束，继续计算该待预测块的运动矢量和运动残差，如果最小值点位于菱形顶点，则继续以此点为中心点，搜索步长和初始搜索步长相同，确定出新的菱形顶点，并继续计算，直到最小值点位于菱形中心点则算法结束。采用此算
法时，搜索点数最小为 13，但是不断用菱形搜索算法持续计算，可能会导致搜索点数很多，而且选用了固定长度的搜索步长，很有可能使算法陷入局部最优，而不是全局最优甚至有可能导致计算发生错误。

因为运动物体的大小和运动情况很是复杂，很难用一种较为简单的算法计算出精确度较高的运动矢量，所以通常将几种搜索算法结合起来进行运动估计，可以结合图像上不同的运动情况，使用相应的算法进行预

测，从而提高预测的有效性、精确度和鲁棒性。

变换和量化

视频中的大部分图像在表达信息的时候有一个相同的特点，平坦的区域和像素值 变化较为缓慢的区域在一帧图像中占据着很大的比例，细化的空间和内容剧烈变化的空间占据的比例较小。从数字信号处理的角度而言，图像在频域上包含着大量的直流和低频成分，而高频分量较少。因此，对图像进行时域到频域的变换，如使用 DCT变换，会使能量集中在图像的左上角区域，使得经过变换后的图像左上角的值较大，而其他区域的值较小，有利于进行熵编码，对数据进行压缩。

H.264 中采用以 4x4 块为运算单元的整数变换算法，此算法在进行变换计算的过程中只有简单的整数系数，这是和之前其他标准所不同的地方。在处理使用帧内编码的图像时进行了分块操作，划分块的最小尺度为 4x4；同时对于帧间编码，在进行运动补偿时对图像进行了分割操作，分割后最小块大小为 4x4，因此整数变换算法采用4x4 块为单元可以准确的对经过帧内核帧间预测后的图像进行变换操作。此变换算法是 Nokia 和 Microsoft 共同提出，使用了新的计算公式。假设输入矩阵为 X，对 4x4块进行变换时方法如下所示：

$$Y = AXA^T = \begin{pmatrix} a & a & a & a \\ b & c & -c & -b \\ a & -a & -a & a \\ c & -b & b & -c \end{pmatrix} X \begin{pmatrix} a & b & a & c \\ a & c & -a & -b \\ a & -c & -a & b \\ a & -b & a & -c \end{pmatrix} \qquad (2\text{-}2)$$

在上式中： $a = \frac{1}{2}, b = \sqrt{\frac{1}{2}}\cos\ \theta(\frac{\pi}{8}), c = \sqrt{\frac{1}{2}}\cos\ \theta(\frac{3\pi}{8})$

如果使用公式 2-2 直接进行运算，则运算系数为无理数，增加了计算难度，且使用软件实现算法时无法使得块经过变换和反变换的结果一致，会降低压缩的准确性。

因此对变换矩阵进行处理以方便计算，对矩阵系数进行四舍五入和部分列进行放大后得出新的变换矩阵 C

$$C=\begin{pmatrix}1 & 1 & 1 & 1\\ 2 & 1 & -1 & -2\\ 1 & -1 & -1 & 1\\ 1 & -2 & 2 & -1\end{pmatrix} \qquad (2\text{-}3)$$

新的变换矩阵中所有的系数均为整数且值为 1 和 2，对于系数为 2 的运算可以用 移位操作替代乘法操作，从而使得整个变换计算过程没有乘法和除法操作，简化了计算过程，提升了计算速度。在解码的过程中，使用对应的反变换矩阵，同样的反变换操作也没有包含乘法和除法操作，且在运算中没有无理数，整个反变换过程也更加简单和快速。

经过变换后的数据值较大，通过量化的方式，可以缩小变换后数据的值范围，方 便随后的编码操作。量化的目的是在保证图像质量的前提下缩小值范围，从而减少图像的编码长度，提高压缩率。H.264 编码标准中采用的是标量的量化方式，量化器的量化公式如 2-4 所示：

$$FQ = round(\frac{y}{QP}) \qquad (2\text{-}4)$$

公式中，y 为输入样本点的编码值，QP 是量化步长，round 函数是取整数值的函数， FQ 为量化结果。可以看出 QP 的值选取的越大，FQ 的取值范围越小，此时经过反量化后能够恢复出的像素细节较少；QP 取值越小，FQ 的取值范围就会越大，此时经过反量化后能够恢复出较多的像素细节，但是在进行后续的编码操作时，编码需要的位
数就越多，压缩率相比于前者降低。因此需要对量化步长和图片质量之间进行权衡，以达到更优秀的编码性能。

去块滤波

H.264 算法中的操作是以块为单位进行计算的，这样的方式对于块内数据而言， 不会造成影响，但是对于块与块之间的边界处理不够精确，从而引入块间数据的误差，使得经过编解码器反变换和量化之后，可以看到图像上有块状边界，即方块效应。产生方块效应的原因主要有两个，其中最为主要的原因是对于帧内数据经过预测后产生
的残差进行 DCT 变换时是基于块进行操作的，帧间预测时对残差数据的处理也是如此。对于变换系数进行量化操作的过程较为粗糙，因此在进行反量化操作时，恢复出的原始数据不够准确，从而导致肉眼看图像时有较为明显的分块边界。另外一个原因是运动补偿，由于运动补偿使用的块在匹配精度上不够准确，当块与块之间有错位时，
边界会出现不连续的情况，参考帧中边界的不连续状态也被引入到进行运动补偿操作的图像中。变换操作是基于 4x4 块的，尺度较小，在一定程度上减轻了方块效应，但是仍然需要专门的算法策略以消除方块效应。

去块滤波[27]的方法主要有两种：一种是添加后置滤波器，另外一种是使用环路滤波器。两者对数据的处理范围有区别，后置滤波器的数据处理范围局限于编码环路之外的位于显示缓冲器中的图像数据，也因此它在标准中只是可以选择的一种选项。环路滤波器用于处理位于编码环路中的数据。在编码器中，经过滤波操作的图像数据被用于作为其他帧进行运动补偿的参考帧；在解码器中，经过滤波操作的图像数据被显示出来。

在视觉上的块效应，不仅是由算法中的误差引起的，当 DCT 变换块的边界，刚好位于图像上的边界，比如器具的边缘等，如果不对此种情形进行特殊处理而认定为是方块效应，就会引入新的处理错误，影响视频质量。所以在对方块效应进行处理的时候，首先要确定边界是否是由算法引入的即假边界还是真实的图像边界，对于真实 边界则不进行处理，对于假边界需要根据残差块的特性和编码模式而使用不同的强度进行滤波。

滤波过程主要分为两大步骤：

1.通过计算得出残差块边界在水平方向和垂直方向上的滤波强度 Bs 如图 2.7。滤波强度 Bs 的取值范围为 0–4，对于亮度块的边界，4x4 块的边界强度值和边界自身的特性有关联，如下图所示。对于色度块的边界，它的滤波强度 Bs 不需要单独进行计算，直接使用对应的亮度块滤波强度 Bs 即可。

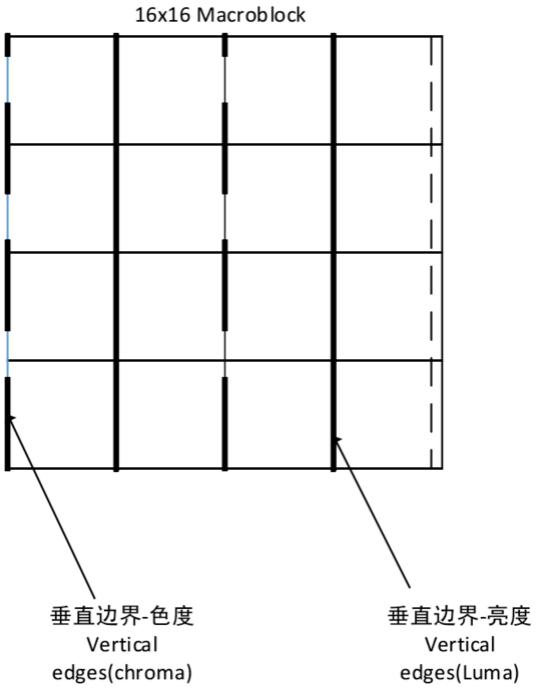


图 2.7 滤波垂直边界示意图

2.为了判断边界是真实边界还是伪边界，确定边界上两边的像素值，根据 1 中确 定的边界强度使用相对应的公式进行滤波操作。由于去块滤波的过程中不对真实边界进行滤波，对由于 DCT 等操作形成的伪边界进行滤波。为了保证图像的视觉效果，应该在尽量去除方块效应的同时保证真实边界不进行滤波。针对这两种情况，分析要进行滤波操作的边界两侧的像素值，即采用自适应样点级滤波。滤波方式主要分为两种：特定的滤波方式用于对滤波强度为 4 的边界进行强滤波；滤波强度为 1、2、3的普通滤波适用于其他情形。

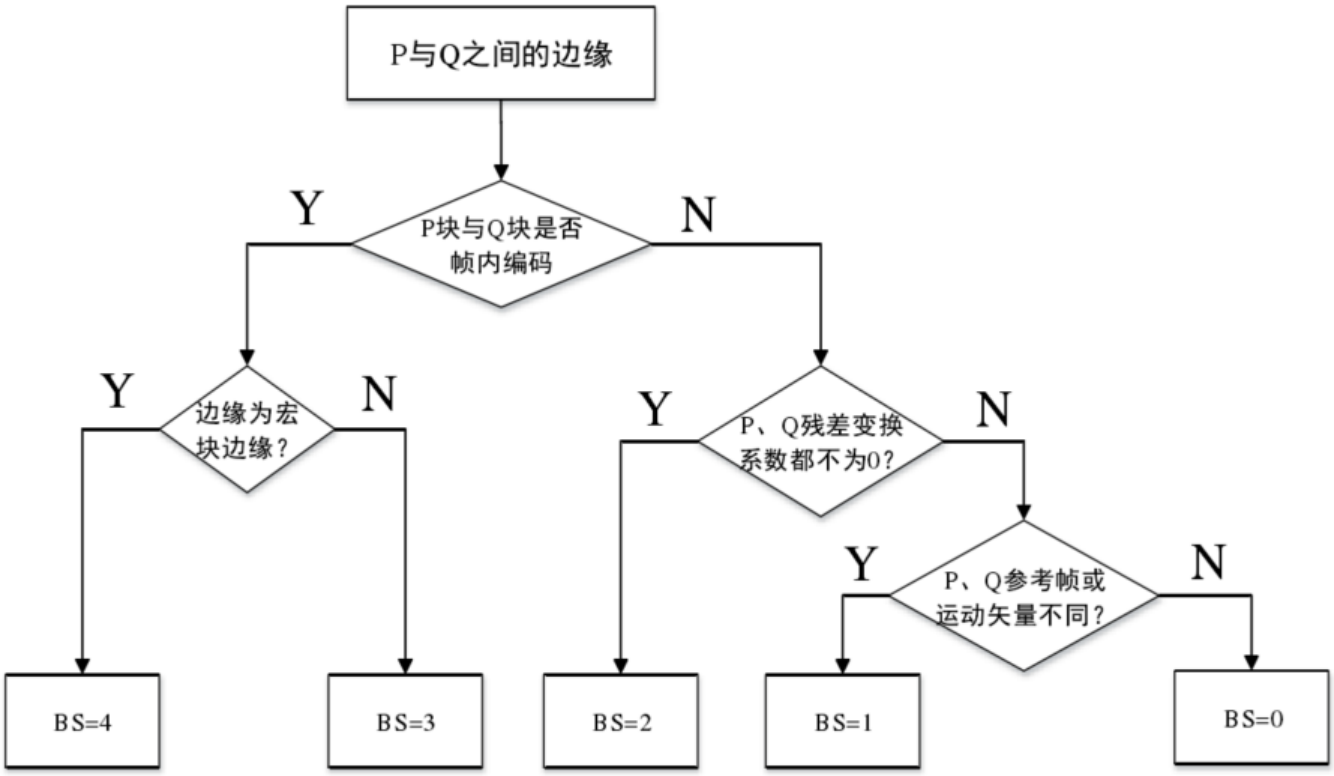


图 2.8 边界强度判断流程图

熵编码

熵编码[28]是利用数据之间的相关性进行数据压缩的一种编码方式，这种无损压缩 编码方法，分为变长编码和算术编码两大类。由于熵编码的无损特点，可以在消除视频数据的统计冗余信息后，再通过编码的方式没有失真地还原出之前的码流信息，能够很好地提升压缩效率，而不影响视频质量。在 H.264 中采用的熵编码方式主要有两种，一种是基于上下文自适应变长编码（CAVLC）和基于上下文的自适应二进制算术编码（CABAC）。由于两种算法的复杂度和特性不同，H.264 的基本档次和扩展档次只支持 CAVLC，而在主要档次中可以使用 CABAC 和 CAVLC 的编码方式。

变长编码的方式是首先统计符号出现的概率，对于出现概率大的字符，使用较短的二进制表示，对于出现概率小的字符，使用较长的二进制表示，使用此种方法大大较少了原始信息的数据量，起到了压缩信息的

作用。算术编码的方式不同于前者，是使用一个浮点数来代替原始数据中的一连串数据，在计算之后，输出的是一个在 0-1
区间内的浮点数。

H.264 中，使用 CABAC 带来的压缩比要高于使用 CAVLC，这一优势随着量化步长 QP 的增加继续扩大，但是压缩比的提升是以算法复杂度的增加为代价的，算法计算所需要的时间比 CAVLC 算法要长，因而具体选用哪种算法进行计算，需要权衡。

H.264有四种画质级别

分别是baseline, extended, main, high：

- 1、Baseline Profile：基本画质。支持I/P 帧，只支持无交错（Progressive）和CAVLC；
- 2、Extended profile：进阶画质。支持I/P/B/SP/SI 帧，只支持无交错（Progressive）和CAVLC；(用的少)
- 3、Main profile：主流画质。提供I/P/B 帧，支持无交错（Progressive）和交错（Interlaced），也支持CAVLC 和CABAC 的支持；
- 4、High profile：高级画质。在main Profile 的基础上增加了8x8内部预测、自定义量化、 无损视频编码和更多的YUV 格式；

H.264 Baseline profile、Extended profile和Main profile都是针对8位样本数据、4:2:0格式(YUV)的视频序列。在相同配置情况下，High profile（HP）可以比Main profile（MP）降低10%的码率。 根据应用领域的不同，Baseline profile多应用于实时通信领域，Main profile多应用于流媒体领域，High profile则多应用于广电和存储领域。

一些解释：

1. I-帧：也成为关键帧，I-帧完全自我使用的，并且不使用任何其他帧的信息。它在三种帧中占最大的比例，并且具有最高的质量，但是压缩效率是最低的。

   P -帧：P -帧是所谓的“预示”帧。当创建了一个P-帧时，编码器可以向后查看I-帧或者P-帧中冗余的图片信息。P-帧比I-帧效率高，但是没有B-帧的效率高。

   B-帧：B-帧是双向预测帧，这意味着当我们创建B-帧，编码器可以同时向前和向后查找冗余的图片信息。这使得B-帧在三种帧中具备最佳的效率。注意，B-帧在使用Baseline方式生产视频的时候是不可用的。
2. 虽然结果是取决于源文件质量的，但通常来说CABAC被认为比CAVLC效率高5%-15%。 这意味着，CABAC应该在码率低5-15%，的情况下，提供同等的，或者更高的视频质量。
3. 至于Baseline@L x.x、Main@L x.x、High@L x.x形式则是在不同级别下的码流级别，数值越大码流就越大，更耗费资源。所以就码流而言High@L3.0<High@L4.0<High@n L5.1。

4. 压缩率来说就是BP>HP>MP，比例大概是100的原始数据， 压缩后BP=15，HP=45，MP=50；在相同配置情况下，High profile（HP）可以比Main profile（MP）节省10%的码流量，比MPEG-2 MP节省60%的码流量，具有更好的编码性能。根据应用领域的不同，Baseline profile多应用于实时通信领域，Main profile多应用于流媒体领域，High profile则多应用于广电和存储领域。

下图清楚的给出不同的profile&level的性能区别。

profile 主要参数：

	Baseline	Extended	Main	High	High 10	High 4:2:2	High 4:4:4 Predictive
I and P Slices	Yes	Yes	Yes	Yes	Yes	Yes	Yes
B Slices	No	Yes	Yes	Yes	Yes	Yes	Yes
SI and SP Slices	No	Yes	No	No	No	No	No
Multiple Reference Frames	Yes	Yes	Yes	Yes	Yes	Yes	Yes
In-Loop Deblocking Filter	Yes	Yes	Yes	Yes	Yes	Yes	Yes
CAVLC Entropy Coding	Yes	Yes	Yes	Yes	Yes	Yes	Yes
CABAC Entropy Coding	No	No	Yes	Yes	Yes	Yes	Yes
Flexible Macroblock Ordering (FMO)	Yes	Yes	No	No	No	No	No
Arbitrary Slice Ordering (ASO)	Yes	Yes	No	No	No	No	No
Redundant Slices (RS)	Yes	Yes	No	No	No	No	No
Data Partitioning	No	Yes	No	No	No	No	No
Interlaced Coding (PicAFF, MBAFF)	No	Yes	Yes	Yes	Yes	Yes	Yes
4:2:0 Chroma Format	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Monochrome Video Format (4:0:0)	No	No	No	Yes	Yes	Yes	Yes
4:2:2 Chroma Format	No	No	No	No	No	Yes	Yes
4:4:4 Chroma Format	No	No	No	No	No	No	Yes
8 Bit Sample Depth	Yes	Yes	Yes	Yes	Yes	Yes	Yes
9 and 10 Bit Sample Depth	No	No	No	No	Yes	Yes	Yes
11 to 14 Bit Sample Depth	No	No	No	No	No	No	Yes
8x8 vs. 4x4 Transform Adaptivity	No	No	No	Yes	Yes	Yes	Yes
Quantization Scaling Matrices	No	No	No	Yes	Yes	Yes	Yes

Separate Cb and Cr QP control	No	No	No	Yes	Yes	Yes	Yes
Separate Color Plane Coding	No	No	No	No	No	No	Yes
Predictive Lossless Coding	No	No	No	No	No	No	Yes
	Baseline	Extended	Main	High	High 10	High 4:2:2	High 4:4:4 Predictive

Level 主要参数：

Level number	Max macroblocks per second	Max frame size (macroblocks)	Max video bit rate (VCL) for Baseline, Extended and Main Profiles	Max video bit rate (VCL) for High Profile	Max video bit rate (VCL) for High 10 Profile	Max video bit rate (VCL) for High 4:2:2 and High 4:4:4 Predictive Profiles	Examples for high resolution @ frame rate (max stored frames) in Level
1	1485	99	64 kbit/s	80 kbit/s	192 kbit/s	256 kbit/s	128x96@30.9 (8) 176x144@15.0 (4)
1b	1485	99	128 kbit/s	160 kbit/s	384 kbit/s	512 kbit/s	128x96@30.9 (8) 176x144@15.0 (4)
1.1	3000	396	192 kbit/s	240 kbit/s	576 kbit/s	768 kbit/s	176x144@30.3 (9) 320x240@10.0 (3) 352x288@7.5 (2)
1.2	6000	396	384 kbit/s	480 kbit/s	1152 kbit/s	1536 kbit/s	320x240@20.0 (7) 352x288@15.2 (6)
1.3	11880	396	768 kbit/s	960 kbit/s	2304 kbit/s	3072 kbit/s	320x240@36.0 (7) 352x288@30.0 (6)
2	11880	396	2 Mbit/s	2.5 Mbit/s	6 Mbit/s	8 Mbit/s	320x240@36.0 (7) 352x288@30.0 (6)
2.1	19800	792	4 Mbit/s	5 Mbit/s	12 Mbit/s	16 Mbit/s	352x480@30.0 (7) 352x576@25.0 (6)
2.2	20250	1620	4 Mbit/s	5 Mbit/s	12 Mbit/s	16 Mbit/s	352x480@30.7(10) 352x576@25.6 (7) 720x480@15.0 (6) 720x576@12.5 (5)
3	40500	1620	10 Mbit/s	12.5 Mbit/s	30 Mbit/s	40 Mbit/s	352x480@61.4 (12) 352x576@51.1 (10) 720x480@30.0 (6) 720x576@25.0 (5)
3.1	108000	3600	14 Mbit/s	17.5 Mbit/s	42 Mbit/s	56 Mbit/s	720x480@80.0 (13) 720x576@66.7 (11) 1280x720@30.0 (5)
3.2	216000	5120	20 Mbit/s	25 Mbit/s	60 Mbit/s	80 Mbit/s	1280x720@60.0 (5) 1280x1024@42.2 (4)
4	245760	8192	20 Mbit/s	25 Mbit/s	60 Mbit/s	80 Mbit/s	1280x720@68.3 (9) 1920x1088@30.1 (4) 2048x1024@30.0 (4)
4.1	245760	8192	50 Mbit/s	50 Mbit/s	150 Mbit/s	200 Mbit/s	1280x720@68.3 (9) 1920x1088@30.1 (4) 2048x1024@30.0 (4)
4.2	522240	8704	50 Mbit/s	50 Mbit/s	150 Mbit/s	200 Mbit/s	1920x1088@64.0 (4) 2048x1088@60.0 (4)

5	589824	22080	135 Mbit/s	168.75 Mbit/s	405 Mbit/s	540 Mbit/s	1920x1088@72.3 (13) 2048x1024@72.0 (13) 2048x1088@67.8 (12) 2560x1920@30.7 (5) 3680x1536/26.7 (5)
5.1	983040	36864	240 Mbit/s	300 Mbit/s	720 Mbit/s	960 Mbit/s	1920x1088@120.5 (16) 4096x2048@30.0 (5) 4096x2304@26.7 (5)
Level number	**Max macroblocks per second**	**Max frame size (macroblocks)**	**Max video bit rate (VCL) for Baseline, Extended and Main Profiles**	**Max video bit rate (VCL) for High Profile**	**Max video bit rate (VCL) for High 10 Profile**	**Max video bit rate (VCL) for High 4:2:2 and High 4:4:4 Predictive Profiles**	**Examples for high resolution @ frame rate (max stored frames) in Level**

从压缩比例来说，baseline< main < high ，对于带宽比较局限的在线视频，可能会选择high，但有些时候，做个小视频，希望所有的设备基本都能解码（有些低端设备或早期的设备只能解码baseline），那就牺牲文件大小吧，用baseline。自己取舍吧！

苹果的设备对不同profile的支持。

iOS Compatability (⇨source)			
Profile	**Level**	**Devices**	**Options**
Baseline	3.0	All devices	`-profile:v baseline -level 3.0`
Baseline	3.1	iPhone 3G and later, iPod touch 2nd generation and later	`-profile:v baseline -level 3.1`
Main	3.1	iPad (all versions), Apple TV 2 and later, iPhone 4 and later	`-profile:v main -level 3.1`
Main	4.0	Apple TV 3 and later, iPad 2 and later, iPhone 4s and later	`-profile:v main -level 4.0`
High	4.0	Apple TV 3 and later, iPad 2 and later, iPhone 4s and later	`-profile:v high -level 4.0`
High	4.1	iPad 2 and later, iPhone 4s and later, iPhone 5c and later	`-profile:v high -level 4.1`
High	4.2	iPad Air and later, iPhone 5s and later	`-profile:v high -level 4.2`

preset

序	name	名称	b-adapt	bframes	direct	me	merange	partitions	rc-lookahead	ref	subme	trellis	weightp	other
1	ultrafast	极快	0	0		dia		none	0	1	0	0	0	no-8x8dct aq-mode0 no-cabac no-deblock no-mbtree no-mixed-refs scenecut0 no-weightb
2	superfast	超快				dia		i8x8,i4x4	0	1	1	0	1	no-mixed-refsno-mbtree
3	veryfast	很快							10	1	2	0	1	no-mixed-refs
4	faster	较快							20	2	4		1	no-mixed-refs
5	fast	快							30	2	6		1	
6	medium	中												
7	slow	慢	2		auto	umh			50	5	8			
8	slower	较慢	2		auto	umh		all	60	8	9	2		
9	veryslow	很慢	2	8	auto	umh	24	all	60	16	10	2		
10	placebo		2	16	auto	tesa	24	all	60	16	11	2		slow-firstpass no-fast-pskip

· --bframes <**最多连续放多少B帧**>

B帧是压缩程度最高的帧，放的越多愈能降低码率！

比如设置--bframes 16 (最大值16)就等于一组最多可以连续放入16个.

注意: 此参数在不同用途的视频的分歧很大.

注意: 此参数会严重影响网络视频的观看效果.

B帧本身需要读取参考前面的帧的信息, 调的越高, 一个GOP中就有更大可能被放入更多B帧从而达到压缩效果.

大量B帧能达到压缩效果, 加快网络缓冲, *但上传用视频占用太多CPU会导致网络播放器从轻微到严重的磕磕绊绊断断续续的播放状态, 这种情况不要设高!*

· --b-adapt <**应该放B帧还是P帧**>

x264用来决定放P帧还是B帧更好, 一共3个选项:

0 全部设B帧 (可能会造成播放器解码困难, 不推荐)

1 加速算法 (bframes开的越大, 速度愈快, 但效果不如2)

2 完整算法 (bframes开的越大, 速度愈慢, 但压缩率更好)

推荐2, 但码率允许的情况下也推荐1

· --direct <**根据空间还是时间来判断物件移动**>

直接 --direct auto 自动判断 (有的参数确实没啥好说的)

· --me <**位移预测方式**>

速度从低到高有:

dia菱形(快, 效果差),

hex六边形(快, 效果一般, 纯单向移动推荐),

umh可变六边形(适中,效果好, 推荐),

esa全向(慢, 效果好点儿)

tesa超全向(比上一个慢点儿, 效果好一丢丢).

大多情况推荐 --me umh

模拟钢琴, midi轨道演示类的视频大多都是单向移动, 所以可以选 --me hex, 但文件码率会增大一点(1080p, 60帧下约100kbps).

tesa只比esa慢一小点. 除非码率只差一点(10kbps)就合格了再使用, 当然更简单粗暴不拖时间的方式还是调 --vf resize 里的 sar~

--merange <**位移预测范围大小**>

多大都可以. 在最极端的 --preset placebo 参数中这个值也只是24.

普通情况就设24即可, 毕竟现在电脑的配置不是几年前了.

模拟钢琴类的视频由于几乎所有的物件都往一个方向移动, 不会很大的拖慢判断, 所以**推荐32**.

--partitions <**宏块能分成多少份**>

用来精确的还原线条. --presetplacebo下的是 --partitions all 所以视频的宽度和高度都会被分成16块, 见图(数一下图中横向和纵向的块的数量就明白了)

推荐:

高清片源+等于/高于1280*720: --partitions all

低清片源/低于1280*720情况下: --partitions b8x8,p8x8,i8x8,i4x4 (IPB帧被分成8x8的小块, I帧还可以被分成4x4的小块.)

当**片源非常不清晰时**: --partitions b4x4,p4x4,i4x4.

--*ref* <**能参考多少帧**> 最大值16

这个参数最终会以命令的形式告诉播放器每一个B帧和P帧能用多少个前面的帧来获取参考信息. 能参考更多帧, 标志着自身可以储存更少信息, 增强压缩效果.

注意: 此参数会严重影响网络视频的观看效果.

****注意: 此参数在不同用途的视频的分歧很大.***

无变化，不用参考，设为1左右。

变化不大，容易参考，适合开大，设为10以上。

变化太大，难以参考，适合缩小，设为5以下。

· --trellis <**要帮忙吗**>

基于拉普拉斯算子(什么鬼)通过分格量化将编码数据微调的参数. 好吧, 拉普拉斯算子的优点是能发现并增强微小的细节, 我就知道这么多~

设定的值会影响trellis在计算部分的参与程度. 一般是越多越好.

0: 关闭.

1: 仅编码后期量化计算.

2: 编码后期量化, 运动精度, 分割宏块等计算.

所有情况推荐 --trellis 2

· --rc-lookahead <**扫描多少帧**>

扫描往后的帧并把变化数据交给量化工具.

建议设置为视频原帧数的三倍, 个位数除以2, 小数去尾.

**对于分辨率很低(240p,360p, 480p等)可以考虑设为视频源帧数的5倍*

(极限值为 250)

并降低qcomp来保证最大的扫描距离和量化压缩强度哦(qcomp后面会讲到, 现在可以无视)

--subme <设置亚像素估计的复杂度>

值设在0-11之间, 设的越大提升越强.

推荐最高的值: 11 (有的x264版本不支持, 这种情况下用10)

默认：7

可使用的值如下：

0 —— fullpel only

1 —— QPel SAD 1 iteration

2 —— QPel SATD 2 iterations

3 —— HPel on MB then QPel

4 —— Always QPel

5 —— Multi QPel + bi-directional motion estimation

6 —— RD on I/P frames

7 —— RD on all frames

8 —— RD refinement on I/P frames

9 —— RD refinement on all frames

10 —— QP-RD (requires —trellis=2, —aq-mode > 0)

preset跟profile，level是否冲突?

就个人理解，profile，是特性限定，level是数量限定，preset是具体执行的步骤限定，在一起应该不冲突。

X264的preset和tune

鉴于x264的参数众多，各种参数的配合复杂，为了使用者方便，x264建议如无特别需要可使用preset和tune设置。这套开发者推荐的参数较为合理，可在此基础上在调整一些具体参数以符合自己需要，手动设定的参数会覆盖preset和tune里的参数。

--preset的参数主要调节编码速度和质量的平衡，有ultrafast、superfast、veryfast、faster、fast、medium、slow、slower、veryslow、placebo这10个选项，从快到慢。

--tune的参数主要配合视频类型和视觉优化的参数，或特别的情况。如果视频的内容符合其中一个可用的调整值又或者有其中需要，则可以使用此选项，否则建议不使用（如tune grain是为高比特率的编码而设计的）。tune的值有：

film：　　电影、真人类型；

animation：　动画；

grain：　　需要保留大量的grain时用；

stillimage：　静态图像编码时使用；

psnr：　　为提高psnr做了优化的参数；

ssim：　　为提高ssim做了优化的参数；

fastdecode：　可以快速解码的参数；

zerolatency：零延迟，用在需要非常低的延迟的情况下，比如电视电话会议的编码。

4.一些编码建议

编码延时

降低x264的延时是可能的，但是会降低质量。若需零延时，设置--tune zerolatency。若你可以接受一点儿小延时(如小于1秒)，最好还是允许延时。下列步骤可以降低延迟，当延迟足够小时，就别再进行后续步骤了：

1.从初始值开始

2.关闭sync-lookahead(设置用于线程预测的帧缓存大小。最大值是250.在第二遍及更多遍编码或基于分片线程时自动关闭)

3.降低rc-lookahead，但别小于10(设定mb-tree位元率控制和vbv-lookahead使用的帧数)

4.降低threads(比如从12降到6)

5.使用切片线程(sliced threads)

6.禁用rc-lookahead

7.禁用b-frames

8.实在不行，就用--tune zerolatency

param->rc.i_lookahead = 0;param->i_sync_lookahead = 0; param->i_bframe = 0; param->b_sliced_threads = 1; param->b_vfr_input = 0;param->rc.b_mb_tree = 0;(使用宏块树位元率控制会改善整体压缩率)

x264线程

x264起多少个线程比较好 ？

建议线程数：

1、2、4、8

测试结论：

1、更多的线程会消耗更多总CPU时间片，因此在长期满载的机器上不宜使用多线程。

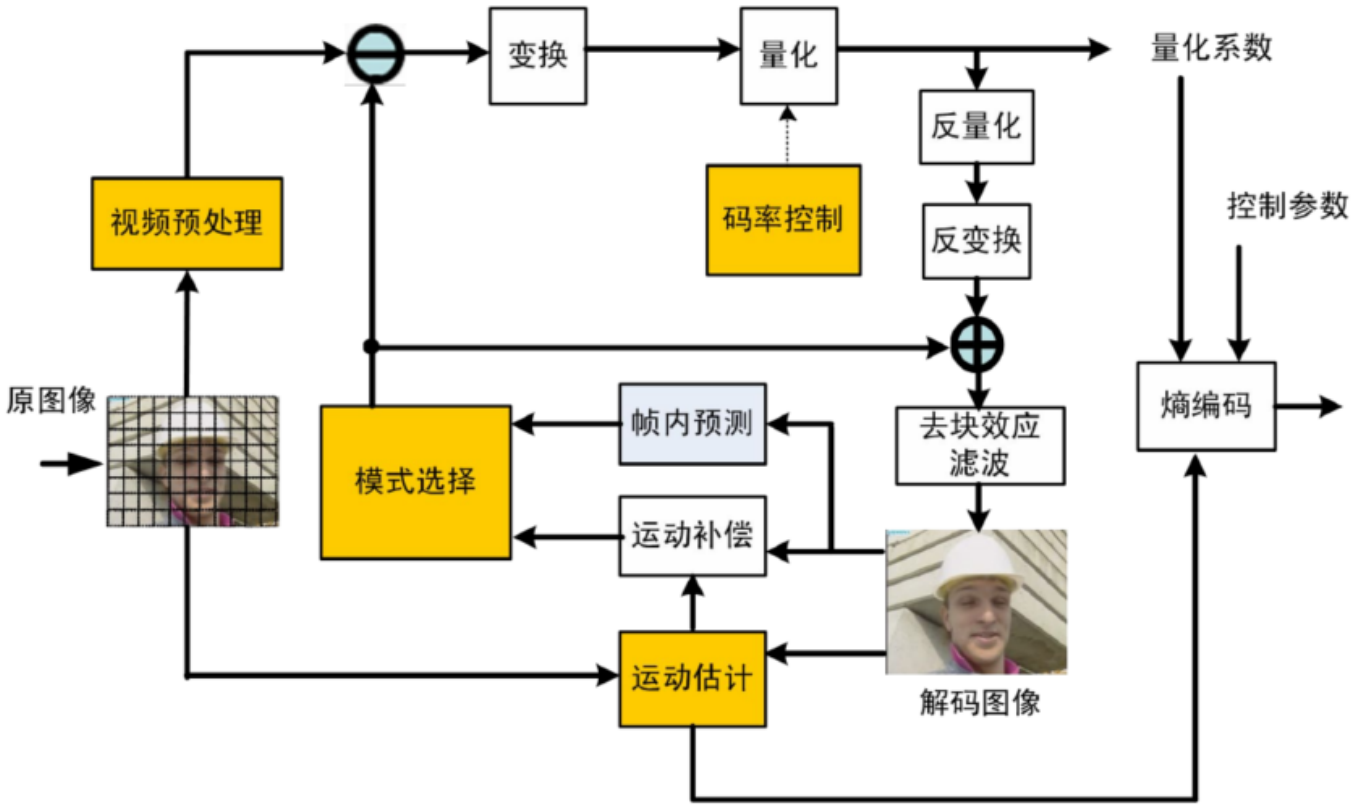
2、获得的时间收益随线程增多呈递减趋势，8线程以后尤为明显。

3、PNSR下降随线程数增加呈抛物递增趋势，16线程增加到24线程PSNR时下降了0.6之巨。

4、设置threads=auto时，线程数为逻辑CPU个数的1.5倍。

编码器的算法可定制模块

H.264 作为工业界广泛使用的视频编码标准，采用了大量编码新技术，使得其具有以下特点：提高了编码效率，与 H.263 等相比，在压缩码率方面提高了约50%，具有较高的压缩性能；能够在一定码率约束下，编码后获取高质量的图像，并且提高传输过程的效率。数据压缩比的提升无疑是技术改进最重要的目标，但同时也伴随着较高的编码复杂度。与其他视频编码标准相同，H.264 是由不同编码技术联合作用，共同完成高效的编码过程，包括场景切换检测、调整图像组、去噪预处理、变换、量化、逆量化、逆变换、环路滤波、运动预测、运动补偿等一系列编码技术。一些技术已在视频编码标准中制定了具体实现方法，但是为了体现视频编码技术的灵活性，只描述了部分编码技术传输的数据流语法内容与编解码流程，允许使用者根据实时编码需求为特定编码技术选取最佳的模式，以满足不同的应用需求[3]。编码参数的不同选项正是编码技术不同模式的体现，根据编码参数在编码流程中的作用，视频编码技术可以划分为不同的算法可定制模块，如图中黄色标记块所示。



编码器端接收原始数字视频图像后，经过对视频复杂度、编码效率与主观视频质量的分析，进行视频图像的处理，这部分即为**视频预处理模块**；

为编码量化模块提供量化步长，控制输出码流的码率，在一定允许条件下，使编码率失真性能达到最优，极大程度的提升视频压缩质量是**码率控制模块**的主要作用[3]；

在视频编码技术中，考虑到视频信号在时间域与空间域的相关性，运动预测技术尤为重要。运动矢量预测的准确程度直接决定了运动预测的效率，这正是**运动估计模块**需要解决的问题；

H.264、AVS 等编码标准中，帧内与帧间预测过程采用了众多编码模式，不同模式的选择影响着编码结果。基于不同视频序列的特征进行模式的选择，是提高编码效率的关键所在，这些工作在模式选择模块开展。

视频编码过程中，**视频预处理、码率控制、运动估计、模式选择**四个算法可定制模块起着决定性作用，协调它们之间的工作，以达到率失真性能、编码复杂度最优化的视频编码算法定制。

（1）视频预处理

视频预处理技术可以根据需要确定是否采用，一般预处理包括下面两种情况：一种是分析视频序列，根据视频内容调整图像组（GOP）结构和编码帧类型，以优化视频编码算法效率；另一种是以提高编码效率和主观视频质量为目标的图像序列去噪处理，以降低视频信号空时域相关性[3]。随着视频编码中多参考帧技术的产生，固定 GOP 结构大小的模式已显逊色，相比之下自适应的 GOP 结构更加灵活，减少了不必要的冗余，虽然增加了编解码器的复杂度，但是可以显著改进预测的准确度。图 2.3 为多参考帧运动预测示例图，H.264 对不同时间的参考帧引入权重，使得在选择参考宏块时扩展到空域和时域两方面，空域上控制得更精准。

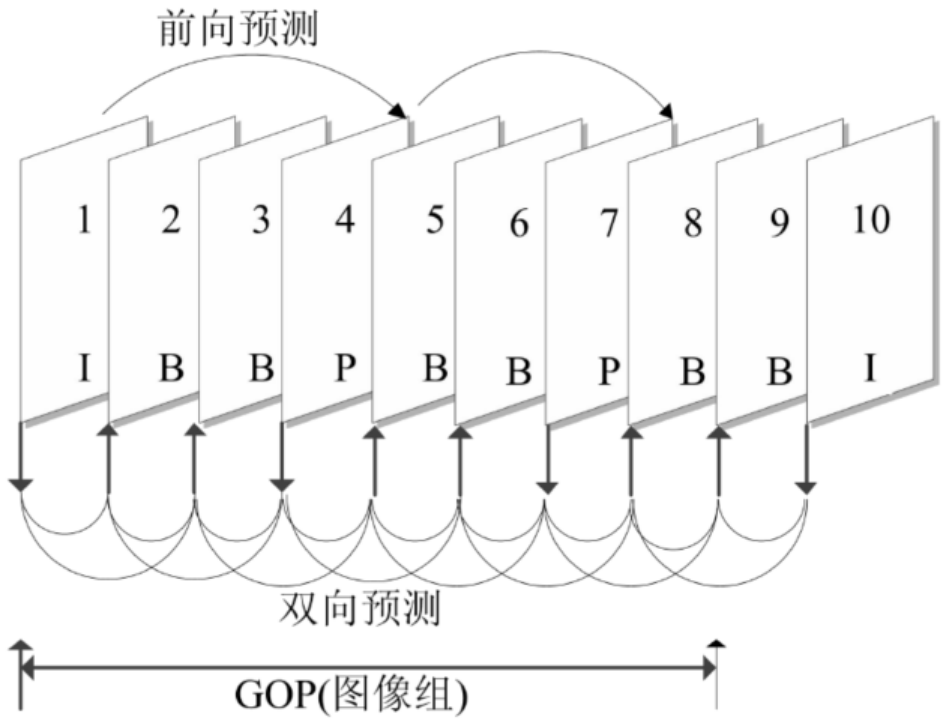


图 2.3 多参考帧运动预测示例图

（2）码率控制

码率控制是视频编码中最复杂的模块之一，任何标准都需要在一定的码率控制约束下执行。码率控制算法是在一定码率约束的情况下，自适应地调节编码器量化参数值，使编码达到优化。MPEG-2 的 TM5、H.263 的 TMN8、H.264、AVC的 G012 等几种码率控制算法已经被推荐作为参考方案。新一代的编码标准HEVC，在码率控制方面也有很大的改进。图 2.4 给出了码率控制模块以及基本原理。

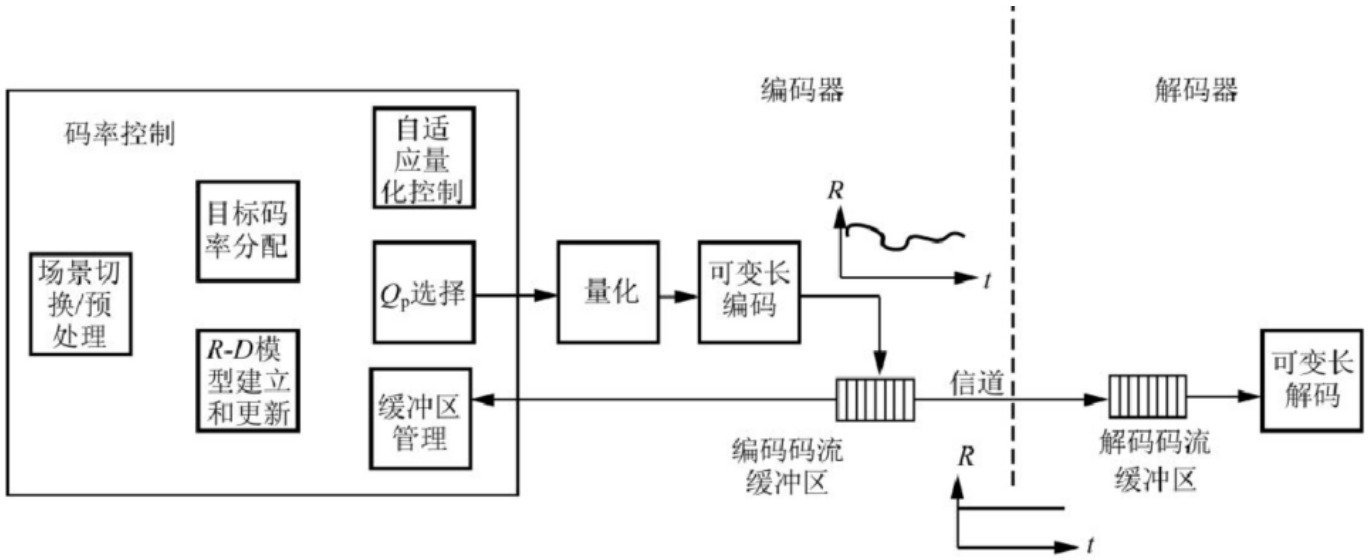


图 2.4 码率控制模块以及基本原理图

（3）运动估计

运动估计模块是视频编码中最为复杂的模块，它的根本目的在于利用视频帧之间存在的相关性，在邻近的多参考帧中进行运动矢量预测，压缩视频信号加以传输。运动估计方法最常用的是块匹配法，将一帧视频图像划分为不同大小的块，加以搜索寻找最优的匹配块。在 H.264 中采用 16x16 固定大小的块，在HEVC中则引入编码单元（CU）这一概念，可以对一个 CU 进行不同模式的分割，最大为 64x64，最小可支持 8x8。匹配搜素算法的好坏很大程度地影响运动估计的精度与实现复杂度，为了降低复杂度，许多学者提出了快速搜索算法，包括：三步搜索算法、共轭搜索法、交叉搜索法、新三步搜索法、四步搜索法、基于块的梯度下降搜索法、菱形搜索法、分层搜索法等，但是快速搜索算法不能保证得到全局最优点。整像素运动估计往往精度不够，因此引用了分像素运动估计。H.264采用的运动补偿精度最少为 1/4 像素精度，也可以选择 1/8 像素精度，HEVC 则采用基于 8 阶 DCT 变换的 1/4 亮度插值和 4 阶 DCT 变换的 1/8 色度插值。使用分像素运动估计与整像素运动估计相比，可减少消耗的比特数[4]

（4）模式选择

H.264 编码标准与其他视频编码标准相比获得较高的编码效率，主要的原因在于模式选择模块的技术改进。帧间运动预测、帧内预测、熵编码技术等等都是模式选择模块引用的先进技术，自适应的模式选择更是提高编码效率的关键所

在。在模式选择过程中不仅要考虑视频图像序列自身的特性，也要考虑场景变更过程中存在的空时域相关性，通过视频的预分析去除一些不需要的模式，这样可以大大降低模式选择过程带来的复杂度。基于率失真优化的模式选择是比较合适的模式选择方法，它的目的是达到率失真性能最优化。同样非率失真优化模式选择的目的是码率、失真、复杂度的折中。
算法可定制模块之间具有一定的相关性。视频序列通过预处理分析，在编码中运动估计与码率控制部分起到作用，码率控制基于视频的预分析获取复杂度分布信息。运动估计模块与模式选择模块也相互影响着进行编码过程的处理，通过模式选择部分不同模式的代价计算，为运动估计提供最优的预测矢量。视频编码中的率失真优化是基于拉格朗日乘子完成的，同时拉格朗日乘子的确定是根据量化参数制定的，因此模式选择与码率控制之间也存在一定的相关性。各个模块间的关系如图 2.5 所示。

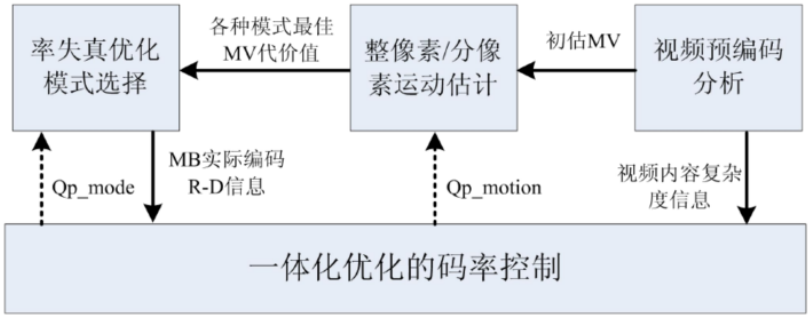


图 2.5 视频编码中模块间相关性示意图

X264编码参数

x264 开源项目作为最优秀的 H.264 编码器，实现了标准中的主要编码算法，其 中包括对不同编码档次的划分、帧内预测的 9 中方向，帧间预测的典型算法如全搜索算法和菱形搜索算法等，对于 4x4 块的 DCT 变换，量化，反变换，反量化以及熵编码等。由于 H.264 中定义的算法太多，x264 工程在保证编码性能不受较大影响的情况下省略了对一些算法的实现。

X264 作为目前工业界广泛使用的 H.264/AVC 编码器，主要进行视频的压缩工作，它的广泛使用基于它可以完成高效率的编码过程。X264 视频编码器的开发到成熟，经历了 10 年左右的时间，这 10 年时间的不断更新使得 X264 的编码性能与效率远远优于其他版本的编码器。与它同期的开源项目 Xvid 采用的早期MPEG-4 标准，X264 压缩的视频文件，在相同的质量情况下小于 Xvid 的压缩文件[6]。
X264 编码器涵盖了 H.264 编码标准中大部分编码技术，是最新的 AVC 编码格式之一。Core 128 是目前最高级的 X264 版本，已经应用于 ffdshow、fmpeg 和MEncoder 等。HM 是新一代视频编码标准 HEVC 的官方标准测试模型，现已进行发布，虽然 HM 采用了 HEVC 中高效率的编码新技术，但是使用仍未得到推广，而是 X264 作为当今优秀的视频编码器在各个应用领域使用。X264 能够提供于视频编码技术研究使用，并且具有很大的利用空间。

编码参数简介

算法可定制模块为了实现先进的编码技术，采用了大量编码参数对编码过程进行控制，包括控制：

- 全局变量参数
- 开关控制参数
- 算法执行参数

这些编码参数共同作用完成了视频图像的高效压缩编码。一些编码参数的设置是编码过程所必需的，对视频编码起着决定性作用；还有部分编码参数，可以根据实际要求人为决定采用与否。针对 X264 视频编码器，算法可定制模块包含的编码参数达几百个之多，主要编码参数如下：

（1）视频预处理模块

bframes

bframes 参数设定 X264 可以使用的**最大并行 B 帧数量**，即设置最多允许几个连续的 P 帧被 B 帧取代。在视频编码过程中，I 帧采用帧内预测方式，P 帧与B 帧类似，采用帧内和帧间预测方式，均可以参考其他帧进行预测，区别在于 P帧只能参考前面的帧进行前向预测，而 B 帧允许以后面的帧作为参考帧进行双向运动预测，如图 2.6 所示，其提高了视频编码压缩效率。Bframes 参数选项范围为 1–16 帧，X264 编码器以 3 帧作为默认选项。

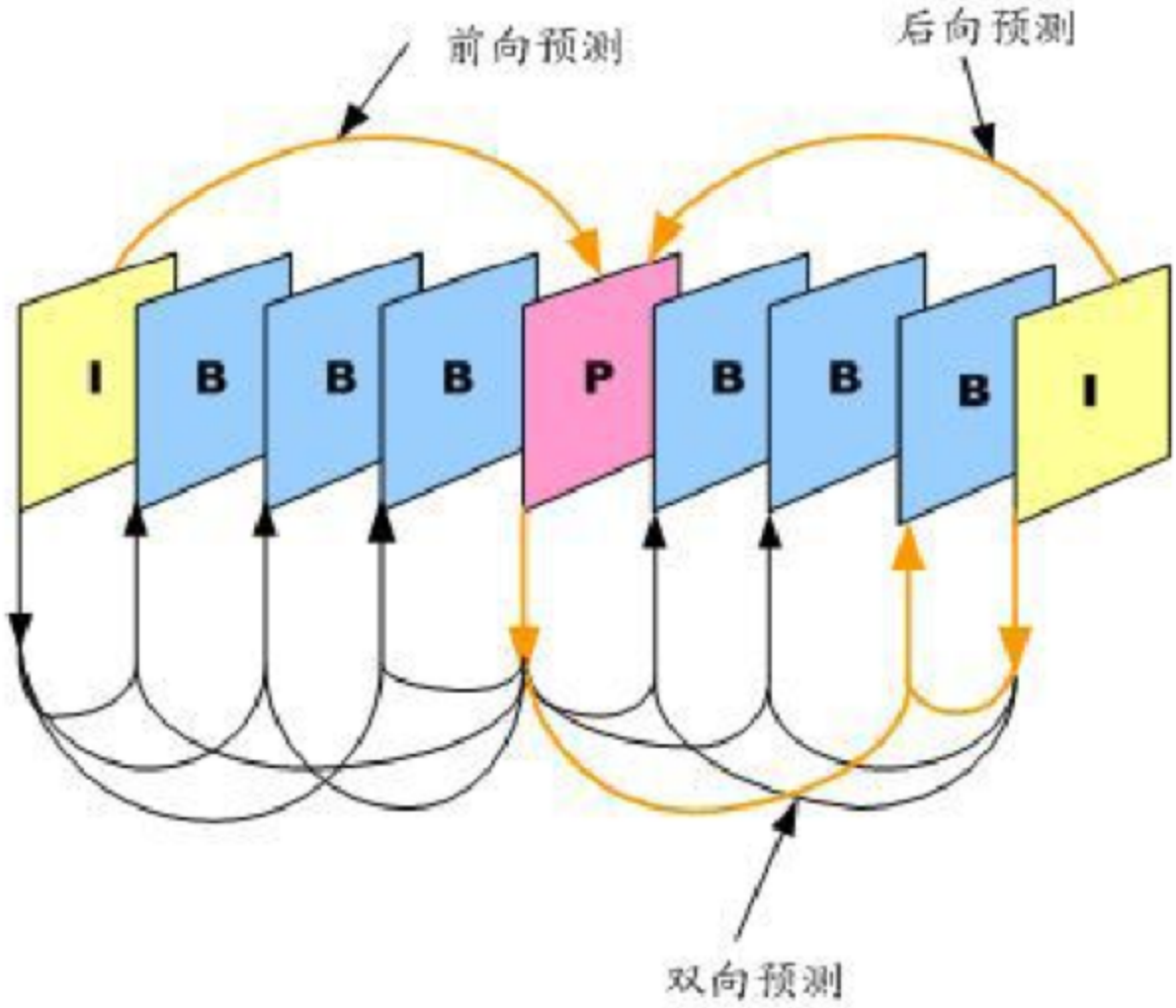


图 2.6 B 帧双向预测结构图

b-adapt

b-adapt 参数设定自适应 B 帧位置决策算法，可以自动调节 P 帧与 B 帧的位置，进行自适应编码优化。在多步编码模式中，此参数仅在第一步中起作用，在第一步结束时编码帧类型已经确定。B-adapt 的选项包括：“0”表示停用，将决策帧全部设定为 B 帧；**“1”表示快速决策算法**；“2”表示最佳决策算法，默认选项为“1”。

b-bias

b-bias 参数是 B 帧决策的衡量值，B-bias 越大，则决策为 B 帧的可能性越大，它与b-adapt 参数协作，完成自适用 B 帧决策。选项范围是-100~100 之间有效整数，但设置为 100 或-100 时，并不保证全部设置为 B 帧，这由算法本身特性决定。“0”为默认选项。

b-pyramid

b-pyramid 参数决定编码过程是否允许 B 帧做为参考帧，关闭此参数，只有I 帧、P 帧可以为参考帧。可以作为参考帧的 B 帧是介于 P 帧和普通 B 帧之间特定的 B 帧，若开启此参数，需保证 Bframes 参数选项大于等于 2，否则将无法生效。选项包括：“none”不允许 B 帧作为参照帧；“strict”表示在一个 GOP

结构中只允许存在一个 B 帧作为参照帧；“normal”表示可以任意使用 B 帧作为参照帧，其中“strict”选项为蓝光光盘编码强制设定选项，默认选项为“normal”。

ref

ref 参数控制解码图像缓存区大小，选项范围 1–16 帧，默认选项为 3 帧。此值的设定决定 P 帧可以使用参考帧的数量，B 帧需要判断是否允许为参考帧。参考帧数量的增多可以提高运动预测的精度，**同时会增加视频编码过程的复杂度。**一般情况下，当参考帧数达到一定数量时，预测精确度并未有显著提升，此参数选项根据实际情况实时分析，参考帧数的增加与性能的提升并非满足线性递增关系。H.264 编码标准限制了不同等级（level）参考帧数目，针对 level4.1 来说，分辨率为 1280x720 的序列（720P）最大参考帧数目为 4，分辨率为 1920x1080的序列（1080P）允许最多参考帧为 9 帧[7]。

keyint

keyint 参数设定 X264 输出文件的最大关键帧间隔。关键帧（IDR）是视频流的“分隔符号”，视频帧无法参考 IDR 帧之前其他视频帧。在 HEVC 中，Keyint参数的设置几乎由 GOP 结构代替，关键帧的设定分为几种情况，包括 GOP 的开启、GOP 关闭等。GOP 关闭时与 H.264 相同，视频帧无法参考前 GOP 中视频帧。

而 GOP 开启时，HEVC 标准中允许当前 GOP 结构中视频帧可以参考其他 GOP中视频帧，同时 GOP 的开启和关闭中增添了多种视频帧设置方式，这方面也是HEVC 编码技术提升的表现之一。X264 中 Keyint 的默认选项为 250，并且在一般情况下，使用者均采用此选项进行编码[7]。

lookahead

Lookahead 参数设定宏块树（Mb-tree）码率控制使用的视频帧数，默认为40 帧。对于 Mb-tree 而言，使用视频帧数越多可以得到更准确地结果，但同时编码复杂度更高。Mb-tree 使用的最大帧数是 Lookahead 参数和 Keyint 参数中较小的值，使用视频帧数的增加能够获得更稳定和精确的码率控制过程。

（2）码率控制模块

qp、bitrate、crf

Qp、Bitrate、Crf 参数，三种码率控制方法。

- Qp 设定 X264 以固定量化值进行编码视频，把某个量化值作为目标，编码后文件大小未知。Qp 固定的编码方式，直接控制着编码图像的失真，失真的大小以及失真的分布特性直接决定着编码视频的主观感知质量；
- Bitrate 以目标码率模式进行编码视频，编码后文件大小已知，但视频压缩质量是未知的。目标码率编码方式，在编码开始时会根据设定的目标码率进行码率分配，依次分为 GOP 级码率分配、帧级码率分配、单元级码率分配，确保复杂度较高的区域分配充足的码率，复杂度较低的区域获取适合的码率，以保证编码失真的均匀分布[8]；
- Crf 是以固定码率系数进行编码，将压缩的视频“质量”作为目标。三种编码方式是互斥的，每个编码过程只能对应一种编码方式。

Aq-mode 参数设定视频编码自适应量化模式。图像由不同空间域矩阵变换时，不进行任何处理其数据是无法减少的[3]。为了视频压缩码率，采用了数据的量化与编码技术，自适应量化是量化技术的提高，动态调节量化步长，优化编码质量。选项“0”表示不启用自适应量化功能。“1”代表编码过程中允许对视频帧以及帧内宏块进行比特数调整，以确保为细节较少的区域分配充足的比特数，为默认选项。“2”可以自动变化 Aq-mode 参数模式，并且可以调整自适应量化的强度，使得自适应的量化编码具有更高灵活性，适用于实验性测试。

Mb-tree 参数控制是否开启宏块树进行码率控制。基于分块预测的视频编码如果出现明显的帧间失真波动，会形成所谓"蚊子效应"，这种时域失真的波动人眼比较容易觉察。因为帧间预测和不同类型帧存在，较小的时域失真波动对于提高预测效率是必须和难免的，但需要控制在一定的范围内，不足以引起人眼明显感知。I 帧和靠近 GOP 头部的 P 帧更多地被后续帧作为预测参考。这些被更多地参考的帧或区域应该有更小的失真，这样可避免失真在 GOP 内累积传播，最大化 GOP 内各帧视频质量。 从这个角度理解，为提高运动预测的效率并有效控制失真波动，需要分析 GOP 中 I、P 帧内各个宏块被参考的频度，在帧内宏块级码率分配中，给被参考频度高的宏块分配相对多一些比特，让有限的比特分配给更重要的区域，提高预测编码效率。X264 采用 Mb-tree 宏块级量化控制算法，基于人类感知特性来动态调节量化参数，改善失真分布情况。Mb-tree 算法主要从以下因素考虑：(1) 作为被参考的宏块，若相邻帧的宏块频繁地参考其进行预测，则应该降低当前宏块的量化步长以减小失真；(2) 由于参考过程中导致失真在传递过程中累加，Mb-tree 宏块树考虑到间接传递产生的影响；(3) 除了客观存在的因素外，考虑到人类感知系统特性，具有复杂场景的区域，即使存在一定失真可能人眼对此并不敏感，而是对平坦区域关注力比较强，因此应降低这部分失真[9][10]。

对于当前视频帧中所有宏块，分别在滑动窗内后续邻近帧中进行运动预测，预测帧的数量由 Lookahead 和 Keyint 两个参数的较小值决定，先分析 B 帧参考和代价传递情况，接着分析 P 帧参考和代价传递情况，依次以树状结构向前扩展，树状结构如图 2.7 所示。Lookahead 分析滑动窗如图 2.8 所示，在编码每一帧之前滑动窗更新一次，得到当前编码帧内每个宏块传递代价。使用 Mb-tree 码率控制会改善整体压缩率，借由追踪跨帧的时域相关性并相应地加权。

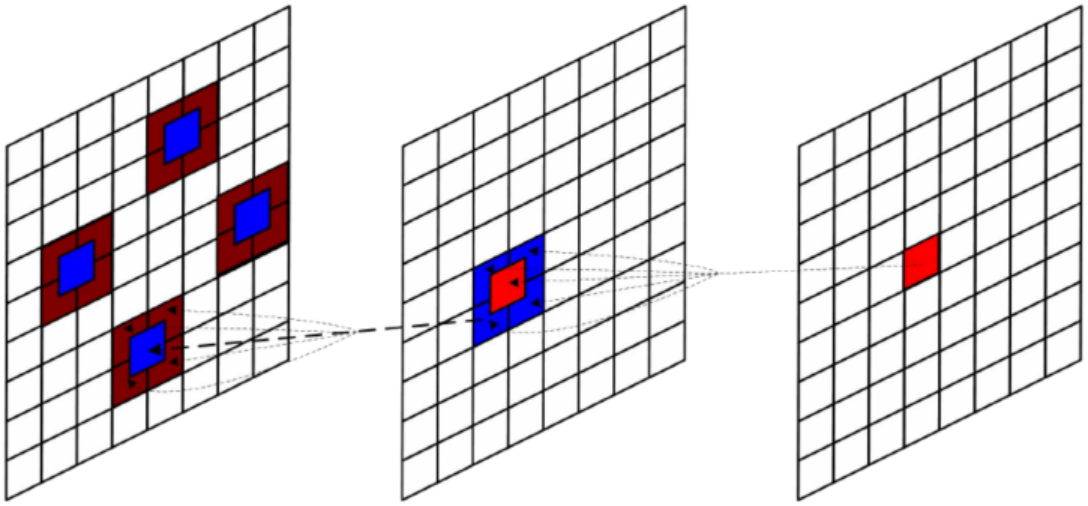


图 2.7 Mb-tree 树状分析结构图

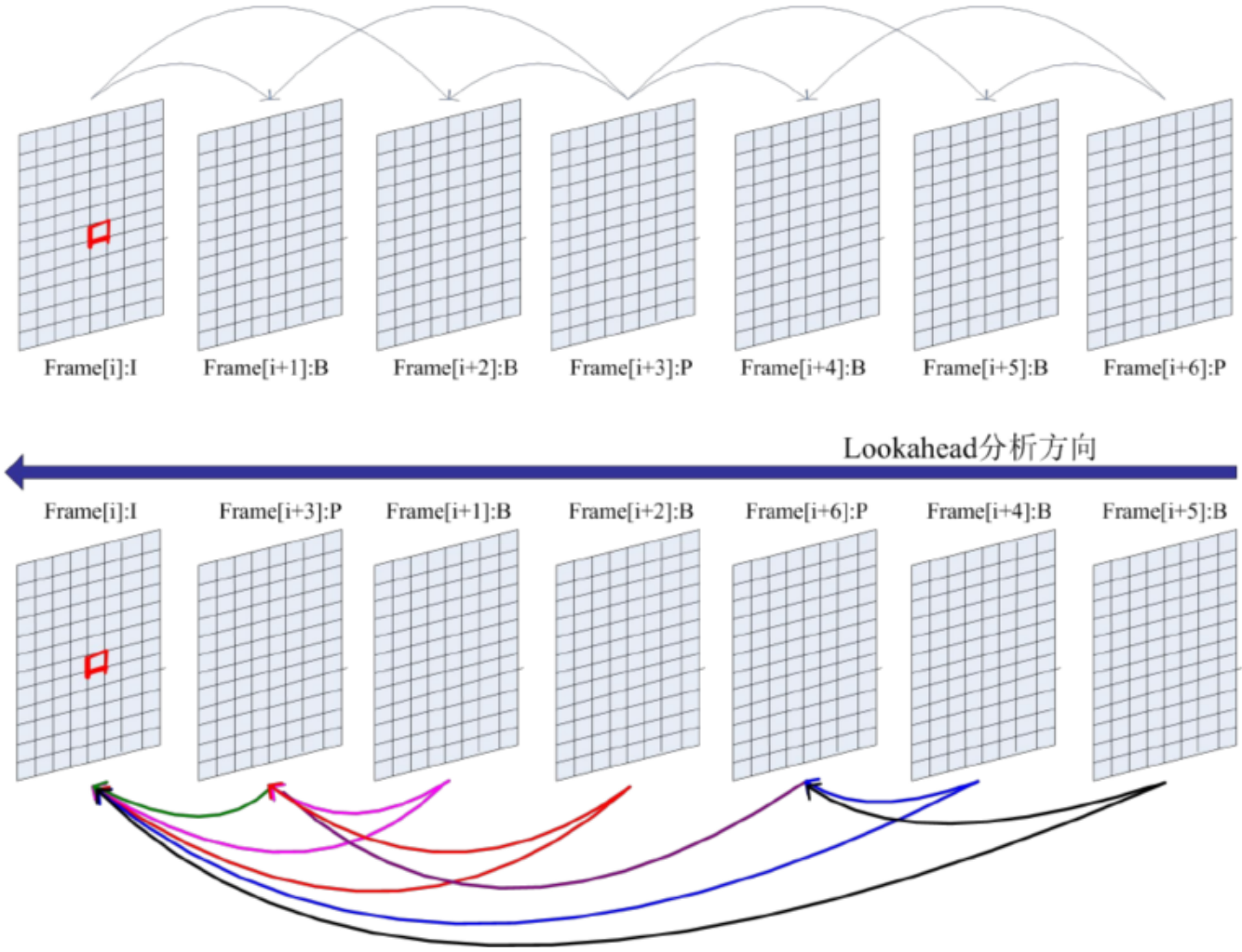


图 2.8 Mb-tree 滑动窗及树状分析结构图

（3）运动估计模块

Me 参数设定全像素运动估计模式，包含五个选项：小菱形搜索（DIA）、正六边形搜索（HEX）、非正六边形搜索（UMH）、全局搜索（ESA）和变换全局搜索（TESA）。DIA 表示最简单的搜寻方法，检查上、左、下、右方一个像素的运动矢量，如图 2.9 所示。重复搜索各点，直至找到最佳匹配点位置，完成搜索；HEX 表示使用周围 6 个范围为 2 的点进行搜寻，因此称为正六边形搜索；此搜索方法效率较高，为默认 Me 参数选项，搜索示意图为 2.10。UMH 表示复杂的多六边形搜索，搜索半径由 Merange 参数控制；ESA 表示一种高度最佳化智慧搜寻方式，搜索范围仍由 Merange 参数限定；

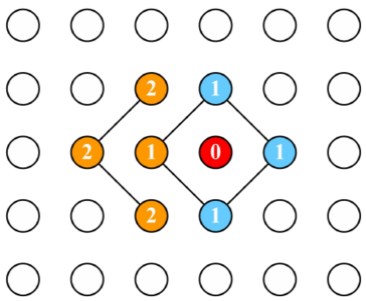


图 2.9 DIA 搜索模版

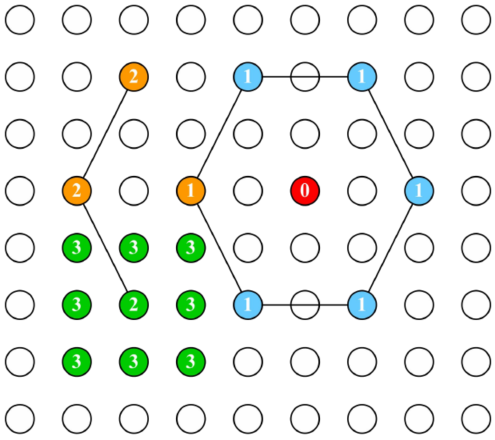


图 2.10 HEX 搜索模版

Merange 参数控制运动矢量搜索的最大范围，单位是像素。设置范围越大，运动估计的复杂度越高。此参数基于运动估计的不同模式进行不同设置，对于运动估计的 DIA、HEX 选项，当设置搜索范围大于 16 时，最大搜索范围仍为 16。

UMH、ESA、TESA 选项允许更大的搜索范围，但加大搜索范围，会大幅度提高编码复杂度。默认选项为 16。

Mvrange 参数设置垂直运动矢量的最大像素值，根据层级的不同，此参数默认选项不同：Level 1/1b–64，Level 1.1–2.0–128，Level 2.1–3.0–256，Level 3.1+–512。

Subme 参数设定分像素运动估计模式。X264 中分像素运动估计分为 11 个层级，层级 1～5 控制分像素细分强度，层级越高，细分强度越大，导致编码过程
越复杂。层级 6 为 I、P 帧的模式决策启用率失真优化（RDO）。层级 7 为全部视频帧启用 RDO，是此参数的默认选项。层级 8 为 I、P 帧的运动矢量和内部预测模式启用 RDO。层级 9 为全部视频帧的运动矢量和内部预测模式启用 RDO。
层级 10 为 Qp 量化开启 RDO，此层级的启用，对自适应量化与 Trellis 参数有一定要求。层级 11 开启所有 RDO，与层级 10 一样需要其他参数支撑。

（4）模式选择模块

Partitions 参数设置视频帧允许的分割模式。H.264 将视频帧在压缩过程中划分为 16x16 的宏块，这些区块可以进一步划分为更小的宏块，完成精确的运动预测。对于 I、P、B 不同类型的视频帧，具有不同划分深度。I 帧可以划分为 8x8、4x4 的区块，P 帧支持 8x8 与 4x4 的划分，B 帧只允许划分成 8x8 的区块。此参数默认选项为 P 帧 8x8，B 帧 8x8，I 帧 4x4。若设定 I、P 帧允许划分 4x4 区块，则表明同样允许划分为 16x16、8x8 区块，B 帧若设定最小可进行 8x8 划分，则仍可进行 16x16 划分。

Trellis 参数执行网格编码量化来提高编码效率，包括三个选项：“0”停用此项功能；“1”只在一个宏块的最终编码上启用此功能，提供了速度和效率间良好的平衡；“2”在所有模式决策上启用此功能，默认选项为“1”。

候选测试编码参数

基于本文提出的显性编码参数标准，对算法可定制模块中重要编码参数进行测评。考虑到编码参数各自的特点属性，候选测试参数以及选项如表 2.1 所示。
Subme 参数的选项“10”与“11”需要其他参数的特定选项支撑，并且应用范围有限，要求较高，并不在本实验考虑范围内。Bframes 与 Ref 参数可支持最大选项为 16 帧，测试选项选定为 1 至 16 的整数。Me 参数的 5 个选项展示了不同的运动搜索方式，均在测试范围内。Keyint、Merange、Lookahead 参数选项范围较大，因此选项选取间隔式测试。B-adapt、Aq-mode、Trellis 参数是编码过程自适应技术的体现，需要在编码过程中开启，不使用“0”选项。
对于模式选择模块的 Partitions 参数，考虑到针对不同帧类型具有不同分割模式，本文引用新组合参数 Sub-partitions 来表示。不同帧类型分割模式的组合，形成了 Sub-partitions 参数的选项“0”“1”“2”“3”，如图 2.12 所示。

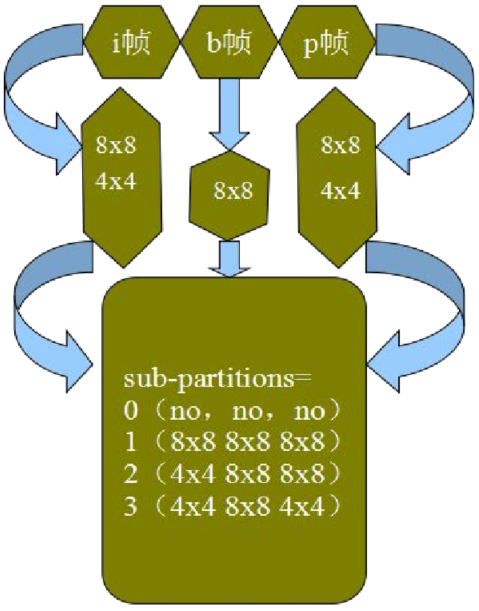


图 2.12 新组合参数 Sub-partitions

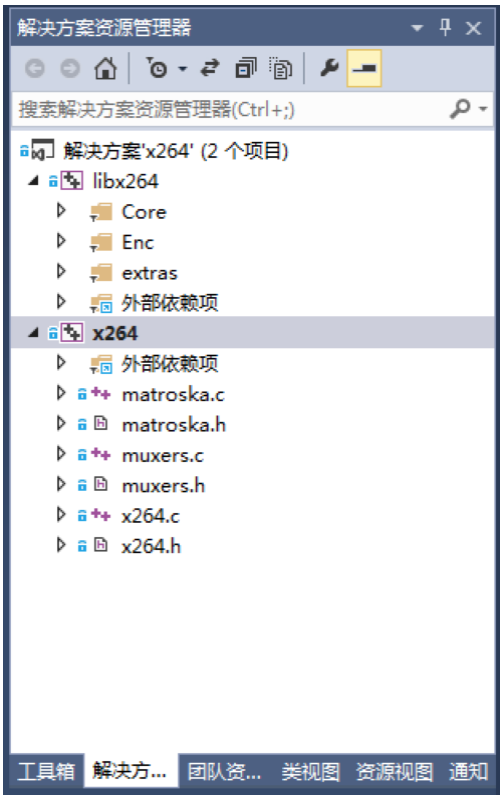
表 2.1 候选测试编码参数及其测试选项

编号	编码参数	测试选项
1	Subme	0，1，2，3，4，5，6，7，8，9
2	Bframes	0~16
3	Ref	1~16
4	Me	DIA，HEX，UMH，ESA，TESA
5	Keyint	30，40，50，70，90，120，150，200，250
6	Merange	4，16，48，64，128
7	B-adapt	1，2
8	Aq-mode	1，2
9	Trellis	1，2
10	Lookahead	20，30，40，50，60
11	Mvrange	32，64，128，256，512
12	Sub-partitions	0，1，2，3

x264 工程分析

x264 项目的实现版本主要由 linux 环境下的实现版本和 windows 平台下的实现版 本。本文在 windows 环境下对 x264 进行优化和移植。使用 VS2013 集成开发环境对x264 工程进行分析。使用此工具打开 x264 工程，有两个工程文件，一个是 libx264工程，另外一个是 x264 工程，libx264 工程的配置类型为静态库（.lib），不包含 main

（）函数等程序启动入口，在 x264 工程中引用了 lib264 工程。libx264 工程中有三个文件夹如图 3.1 所示：



可以看出 libx264 包含四个文件夹 Core、Enc、extras 和外部依赖项，外部依赖项中存放着工程所需要的头文件，比如 pixel.h，dct.h 等，Core 文件包含编码所需要的
核心文件比如 dct.c，quant.c（用于实现 DCT 变换和量化操作）等，Enc 文件夹下存
放着和编码操作密切相关的文件比如 cavlc.c 和 ratecontrol.c(熵编码和码率控制），
extras 文件夹下包含着处理输入参数的文件 getopt.h 和大量的宏定义 stdint.h。x264 工
程中的文件较少，包含控制整体编码流程的 x264.c，其中包含函数的入口点 main（）
函数以及 Encode()函数。

编码流程

x264 项目的执行是从程序入口点 main()开始执行的，下图中为 x264 工程从 main 函数开始的执行流程图：

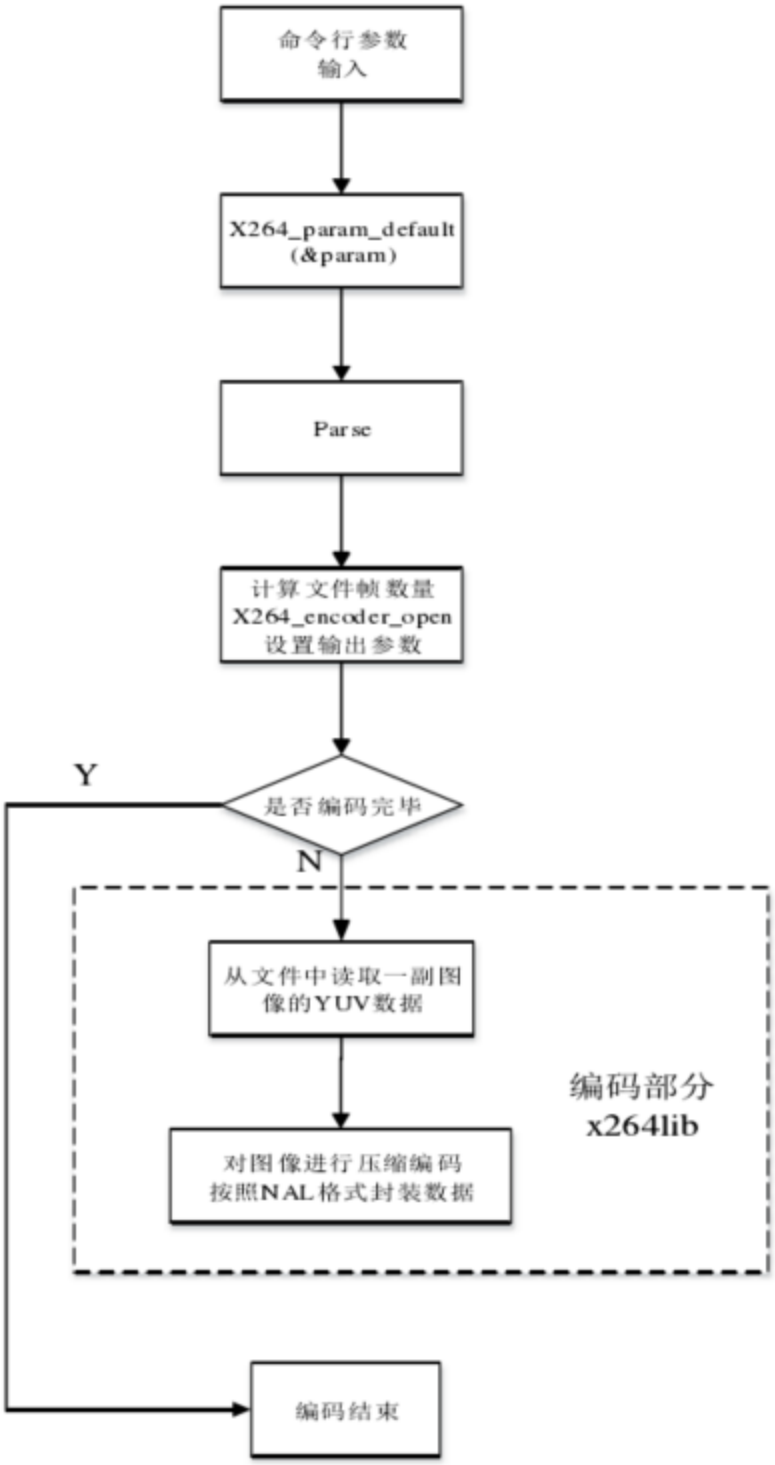


图 3.2 x264 编码流程图

x264 项目中实现了 H.264 标准中的大量算法，对于选用哪种算法进行运算和算 法中的关键参数都需要进行设置，这样才能使项目更加具有灵活性和适应性，针对不同的应用场景和应用需求选择不同的参数如码率、量化参数、参考帧数量和快速优化算法等。x264 中定义了数据结构 Struct x264_param_t,其中定义了和编码器控制相关的大量参数，比如 struct analyse、struct rc（码率控制）、struct vui 以及 i_fame_refence(可 以使用的参考帧最大数量)、i_bfame(两副图像之间可以使用最大的 B 帧数量)、

i_threads(编码中并行处理使用的线程数)、i_width（图像的宽度，以像素为单位）、i_height（图像的高度，以像素为单位）、i_fame_total（视频中需要编码的帧数量）和 i_fps_num（帧率）。
x264 在进行编码之前，通过设置命令行参数，向程序中传入了部分所需要的编码器控制参数。比如：要进行编码的文件位置和经过编码操作后压缩文件的存放位置通过-o 选项设定。首先对编码器控制参数 x264_param_t 进行初始化，使用x264_param_default(&param)函数对控制参数进行默认赋值。随后使用 Parse(argc, argv, &param, &opt) 函数处理从命令行中传入的参数，具体的提取参数操作是由getopt_long(argc, argv, short_options, long_options, NULL)函数完成的，其中 argc 为从命令行输入的参数个数，argv 类型是 char**，为输入的参数的引用。经过此函数处理后将处理后的参数放入 optarg 中并分别于编码器已经设定好的控制参数进行匹配，如果相等则设置为对应的参数值。通过以上操作完成了对编码器的参数设定。

设置完参数后，开始进入编码函数 ret = Encode(&param, &opt)，编码函数的返回值 ret 用来表示编码是否成功。在此函数中首先根据输入的参数对编码器进行初始化x264_encoder_open(param)，并对要编码帧的数量进行了计算。使用x264_picture_alloc(&pic,x264_CSP_I420,param->i_width,param->i_height) 函数创建 一
副图像用于存放视频文件的 yuv 数据，其数据格式为 YUV420。在完成这些初始化操作后，确定了需要编码的总帧数 i_frame_total，以此为控制条件，在被编码帧序号小于 i_frame_total 的状态下，对每一帧原始数据格式图像进行编码。具体而言，先调用p_read_frame(&pic, opt->hin, i_frame + opt->i_seek) 函数从视频文件中读入一个视频帧的数据，放入 pic 指针对应的内存中。然后调用 Encode_frame(h,opt->hout, &pic)对一帧图像 pic 进行编码。编码器对一帧图像进行编码的具体操作是在此函数中完成的。
在 Encode_frame 中的核心函数为 x264_encoder_encode(h, &nal, &i_nal, pic, &pic_out)，对图像的压缩编码实际上是由此函数完成的，除了编码操作外，还对编码后的数据使用 NAL 包格式进行了封装，以便于在网络中传输。x264_encoder_encode的流程图如图 3.3 所示。在此函数中首先对程序使用的线程数量进行判断，如果使用的线程数大于 1 ，则记录当前线程，下一个将被使用的线程和较早被使用的一个线程，随后对线程运行的环境进行初始化，为线程的运行做准备，并在当前线程和下一个将运行的线程不同时，执行参数和资源拷贝指令。然后更新图像帧所需要使用的参考集合。

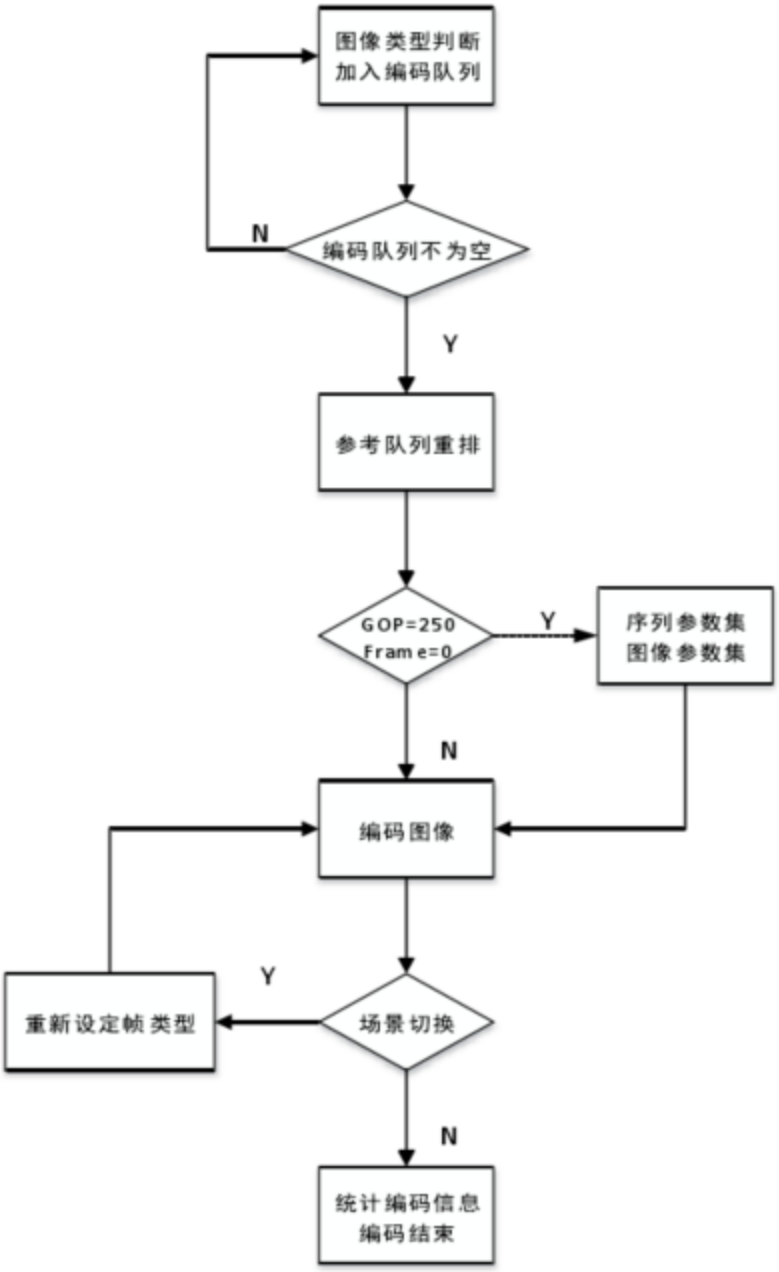


图 3.3 图像帧级别编码流程

在对帧类型进行设定之后，将等待被编码的图像加入编码队列中。如果编码队列 的长度为 0 就需要重新设定参考集合，当队列长度达到 GOP 长度时，也需要采用同样的操作。由于在编码的过程中会使用到 B 帧，所以需要将编码后的数据也保存起来，为随后进行编码的图像提供参考，x264 在实现工程中设置了两个队列，一个为next 队列按照图像在时间上连续的顺序进行存储，另外一个 current 队列是按照已经编码的图像帧的顺序存储。如果按照编码器的设定，在 P 帧之间有 3 个 B 帧，则首先将这些图像帧放入 next 队列中，如果此队列中保存了 4 个图像，便对帧的类型进行判断，通过函数判断出这四个帧的类型为 B、B、B 和 P，对应于代码中的SLICE_TYPE_I、SLICE_TYPE_P 和 SLICE_TYPE_B。由于 P 帧在编码时不能参考 B 帧，而 B 帧的编码需要依赖于 I 帧、P 帧或者 B 帧，并且通过对这四帧图像在 next队列中的位置关系，判定出先对 P 帧进行编码，随后按照时间上连续的顺序将其余三帧加入 current 队列中。如图 3.4 所示：



图 3.4 图像帧队列

其中使用到参考帧，由于在 x264 可以选用的参考帧数量较多，需要使用数据结 构进行维护，于是根据 H.264 协议中的规定，设置两个参考队列用于存放参考帧，分别是 list0 和 list1。第一个队列 list0 用来存放被用于其他帧进行预测参考的前向帧，后一个队列 list1 被用来存放被用于其他帧进行预测参考的后向帧。前向帧还是后向帧的判断依据是 POC 的大小。在 x264 中为了避免在两个参考队列中出现相同参考帧的情形，而导致对同一帧数据进行重复预测的状况，在 list1 中一般只存放一个后向参考帧。

有了之前参考帧的设置和更新方法，就可以对带编码的图像进行帧间预测等操作。
在对图像帧进行编码时，对图像进行了不同层次更小单元的划分，首先划分为 slice，
划分后调用 x264_slice_init(h,i_nal_type,i_global_qp)对条带进行初始化操作，核心功能
是使用 x264_slice_write()函数进行操作的。这一过程中包含了对宏块的分析、对宏块
进行编码、熵编码以及码率控制和去方块滤波。

整个 x264 项目中最耗时的部分就是对宏块进行操作。帧间预测占据工程的预测
部分的主要部分，而运动估计确定带编码块的匹配块，确定运动矢量，以及最后的生
成残差操作是其主要内容。使用 x264_macroblock_analyse(h)函数宏块进行运动估计，
在使用从众多的运动预测算法中选出合适的算法进行预测后生成运动矢量 MV，接下
来使用 x264_macroblock_encode(h)计算出残差数据，使用这个函数时应当注意，在需
要的时候它可以改变宏块的类型至 P_SKIP。由于得出的残差数据仍然有着较强的相
关性以及数值范围较大不利于直接进行编码压缩，所以对残差数据的变换块进行变换，
将时域上的数据转换到频域上，转化之后，数据的分布大多集中在图像的左上角区域，
有利于进行编码操作，然后对变换后的数据进行量化，根据开发者对编码 QP 设置的
要求，采用适当的 QP 将值范围缩小到一定区域，再使用 zig-zag 行程编码的方式组织数据，最后进行重建等操作。
对于运动搜索部分，为了能够得到正确的匹配块和运动矢量，x264 使用了亚像
素细化，提供了多种可供选择的运动搜索算法并且可以设置搜索步长等相关参数，具
有较强的灵活性和适应性，运动搜索的性能对图像质量以及编码器的编码速度都有着
很大的影响，本文在下面的优化部分针对不同的搜索算法进行了研究和测试。

X264优化

由于 x264 强大的功能和适应性，它有着广泛的应用，比如 VLC、FFMPEG 等软件使用了 x264 编码器，在视频监控领域和视频会议等需要进行实时编码的场景中，
x264 无疑是当前使用最多的编码器，厂商们通过对 x264 进行裁剪,将其应用到不同的
环境中。

本文设计的系统采用 x264 作为编码器，在对系统进行优化时，主要是对 x264 编码器进行优化，以提高帧率，增强实时性。经过对 x264 工程的介绍，可以了解到 x264 有着强大的功能以及灵活的控制参数设置，以适用于不同的应用场景和不同用户的需求。在编码器中衡量系统性能的重要指标有 PSNR（视频质量）、码率（反应编码后视频的数据量）和 fps（帧率）等。通过选取不同的控制参数可以使编码器的性能在不同指标上面有所提升，但是这种提升是有代价的，比如通过选用不同的运动估计算法以及不同的编码器控制参数使 PSNR 提升，可能会导致码率的增加。也就是在一个指标上的性能提升可能会导致其他指标上性能的下降，这需要我们进行权衡，而在实际需求中，恰好对某一性能的要求较高，就可以通过此种方式达到优化的目的。当前市场上的一些硬件设备专门对图像处理做了优化，将 x264 算法移植到选定的硬件平台上，就可以达到计算性能的提升，比如专用的图像处理硬件平台 DSP 处理芯片等。本文主要根据自身的需求通过选定合适的编码器参数和使用多线程技术对 x264 进行优化，以提升整体性能。

参数优化

在 x264 编码器中如果不在命令行中对参数进行设置，则会使用默认的参数集合， 对应于程序中的 x264_param_default(&param)。由于本文设计的系统应用需求为实时编码，对实时性要求较高，同时压缩的原始视频要求是 720P，因此采用的档次为 Main Profile，由于 720P 的视频每帧数据量很大，为提高压缩比率使用 I、P、B 帧。在此基础上分析不同参数对系统性能的影响，并最终确定配置参数。**运动估计是 x264 中最为耗时的部分**，提供的运动预测算法也较多，比如菱形搜索、正六边形搜索、可变半径六边形搜索和全搜索等，主要对这一部分进行优化。下面对量化参数 QP、参考 帧数量、搜索方式以及动态预测和分区方式的控制参数进行研究测试，以确定恰当的控制参数来在保证图像质量的情况下提升系统的性能。

进行优化研究的硬件平台和测试序列等软硬件环境如表 3.1 所示，为了显示PSNR 和码率帧率等统计信息，需要加入命令行参数--psnr。

表3.1 测试环境

测试序列	Keiba_416x240_30.yuv	序列内容	赛马
序列分辨率	416x240	总编码帧数	30
测试软件	x264	操作系统	Windows 7 64 位 旗舰版
测试软件平台	Visual studio 2013	CPU	Intel Core i3-2120 CPU @3.30GHz

1. 量化参数研究。在量化过程中实际用于计算的参数是 Qstep 量化步长，测试 使用的是从命令行中传入的控制参数为 QP 量化参数，两者之间有着对应关系，它们都有 52 个值，取值范围为 0-51，取 0 时表示无损输出，QP 的值增加 1 相应的量化步长增长了 12.5%。以下对 QP 设置不同的值，通过 PSNR、码率和编码速度三个指标来查看对系统性能的影响。设置参数的格式为：-q 或者-qp。设置不同 QP 的测试结果见表格 3.2。

表3.2 QP 测试结果

QP	1	10	20	30	40
PSNR（dB）	62.784	51.111	44.321	37.929	29.711
码率（kb/s）	10711.57	4586.31	1275.34	389.92	83.65
编码速度（fps）	19.07	25.21	49.30	68.27	114.02

以下图像组3.5中五张图像的先后顺序对应于QP分别设置不同参数值时的编码图像。



（a）QP 为 1 的编码图像　（b）QP 为 10 的编码图像

（c）QP 为 20 的编码图像　（d）QP 为 30 的编码图像

（e）QP 为 40 的编码图像

图 3.5 测试 QP 时编码图像

由表格中统计出的数据可以得出，对于此视频序列，随着 QP 的增大，PSNR 值 逐步减小，码率相应的降低，但编码速度有相应的提升。但是当 QP 取值在 30 以上时，虽然帧率有较大幅度的提升，QP 为 30 时相对于 20 提升了 1.38，QP 为 40 时相对于 20 提升了 2.32，但是视频的质量下降很明显,。由此得出对 QP 参数的设置对于本文主题实时性有较大的影响，在保证视频质量的前提下可以通过选取适当的 QP，提升编码速度。本文中设置 QP 为 25。

2. 最大参考帧数量研究。在进行帧间预测时，H.264 设定了两个参考帧队列， 用来存放用于进行预测的已经解码过的帧，当使用较少的参考帧时通常意味较少的计算量，但是预测的准确性有所下降，并且可能造成码率的增加。在 x264 中这一选项的参数格式为–r 或者––ref，默认值为 3，取值范围为 0–16。对于它的测试结果如表格3.3 所示：

表3.3 ref 测试结果

参考帧数量	1	4	7	10	13
PSNR（dB）	38.553	38.601	38.642	38.643	38.644
码率（kb/s）	469.83	450.50	437.42	435.56	435.33
编码速度（fps）	97.85	90.83	71.99	58.30	53.50

以下图像组 3.6 中五张图像的先后顺序对应于 ref 设置不同参数值时的编码图像。



（a）ref 为 1 的编码图像　（b）ref 为 4 的编码图像　（e）ref 为 13 的编码图像

（c）ref 为 7 的编码图像　（d）ref 为 10 的编码图像

图 3.6 测试 ref 时编码图像

由表格中的数据可以看出当最大参考帧的数量逐渐增加时，码率和帧率会下降，但是当 ref 值超过一定值时，效果就会显得不明显，同时对 PSNR 的影响几乎没有。
理论上，不同分辨率的视频能够有效使用的参考帧数量是有限的，也和这一测试的结
果符合。不同 ref 对应的编码图像如上图组所示，本文中设置 ref 为 3。

3. 运动估计是 x264 算法中很重要的一部分，通过 x264 命令行可以控制选用不 同的搜索算法。使用不同的搜索算法时会影响运动估计的计算规模，从而会对压缩的准确性和速度造成影响。在 x264 中设置的

格式为--me，可以设置的参数值有表格中的四种，分别是菱形搜索、正六边形搜索、可变半径六边形搜索和全局搜索。其中默
认值为 hex,即采用正六边形搜索。测试结果如表格 3.4 所示：

表3.4 搜索算法测试结果

搜索算法	菱形搜索	正六边形搜索	可变半径六边形搜索	全局搜索
PSNR（dB）	38.565	38.601	38.596	38.600
码率（kb/s）	451.25	450.50	446.94	447.41
编码速度（fps）	93.71	88.43	81.95	63.13

以下图像组 3.7 中四张图像的先后顺序对应于不同搜索算法时的编码图像。



（a）菱形搜索的编码图像　（b）正六边形搜索的编码图像

（c）可变半径六边形搜索的编码图像　（d）全局搜索的编码图像

图　3.7 测试搜索算法时编码图像

由表格中的数据可以看出，对于此测试序列，当选用不同的搜索算法时，帧率会 发生变化，表格从左到右依次增加。但是对码率和 PSNR 的影响较小，使用全局搜索算法时运动估计时计算量最大帧率最小，采用菱形搜索时帧率最高，后者比前者提升了 1.48 倍。因此本文在系统设计时，使用菱形搜索算法，以获得较高的帧率，从而满足实时性要求。

4. 在确定需要使用的搜索算法后，搜索的范围也会影响到算法的计算量和效率， 搜索的精度比如是采用整像素进行搜索或者 1/2 像素，1/4 像素等会影响块匹配的准确性，算法复杂度也不相同，这些会对系统性能造成影响。下面选取不同动态预测和分区方式，查看对编码性能造成的影响。在 x264 中参数选项为--m,一共有 7 个选项，1、用全像素块进行动态搜索，对每个块再使用快速模式进行四分之一精确搜索；2、

用半像素块进行动态搜索，对每个块再使用快速模式进行四分之一精确搜索；3、用半像素块进行动态搜索，对每个块再使用质量模式进行四分之一精确搜索；4、用快速模式进行四分之一像素块精确搜索；5、用质量模式进行四分之一像素块精确搜索；6、进行 I、P 帧像素块的速率失真最优化；7、进行 I、P 帧运动矢量及块内部的速率失真最优化。默认的模式值为 5，表示用质量模式进行四分之一像素块精度搜索。在此对方式 1、3、4、5、6 和 7 这六个选项进行测试，测试结果如表 3.5 所示。

表3.5　动态预测与分区方式测试结果

方式选择	1	3	4	5	6	7
PSNR（dB）	38.341	38.403	38.405	38.434	38.537	38.601
码率（kb/s）	485.04	435.36	429.95	429.55	443.88	450.50
编码速度（fps）	148.57	136.63	132.60	115.11	91.24	78.86

以下图像组 3.8 中六张图像的先后顺序对应于动态预测与分区方式设置不同参数 值时的编码图像。



（a）方式 1 时编码图像



（b）方式 3 时编码图像



（c）方式 4 时编码图像



（d）方式 5 时编码图像



（e）方式 6 时编码图像



（f）方式 7 时编码图像

图 3.8 动态预测与分区编码图像

由表格中数据可以看出，选取不同的动态预测与分区方式，对帧率的影响最大， 对于码率和 PSNR 的影响较小。从图 3.8 中的图像效果也可以看出视频的视觉效果相差很小，与 PSNR 的结果相符合。本文中设置 m 为 1。

上述测试结果表明通过设置量化参数、最大参考帧数量、运动搜素算法和动态预测与分区方式均会对编码速度造成较大幅度的影响；量化参数对 psnr 的影响较大，但是，在实际应用中，不能单纯的追求图像的客观质量，需要根据图像主观上的质量对其进行设定，因此可以通过设置 qp 来提升编码速率；本文主要的设计目的是在保证视频质量的前提下提升编码速率，码率方面不做过高要求。综合使用不同参数对系统性能造成的影响，根据本文的需求进行恰当的设定，可以达到提升编码速率的目的。

并行优化

对于一些计算耗时的任务，可以通过使用多线程等并行处理方式来减少计算时间。x264 项目中支持 slice 层级的并行编码操作，并且支持使用 pthread 来创建多线程[29]，
创建新线程来以 slice 为单元进行压缩编码，最大程度上可以同时对 128 个分片进行编码操作。当多线程数目的设置 i_threads 大于 1 时，使用 x264_pthread_create 创建新的分片进行编码。具体的，在主线程中对目前开启的线程数目进行判断，如果小于 i_threads，thread_oldest 指向新线程，h->b_thread_active 被设置为 0，此时不能执行 x264_encoder_frame_end()函数的相关代码段，主线程继续循环创建 x264_slices_write 线程，当被创建的线程数目达到 i_threads 时，thread_oldest 被指向 i_threads 个线程中被系统判断返回最快的， h_b_thread_active 也被设置为 1， 此时将执行 x264_encoder_frame_end()函数，在此函数中调用 x264_pthread_join()，从而阻塞主线程，直到 thread_oldest 完成后才重新创建新线程，通过此种方式，保证对编码部分进行多线程处理并且运行的编码线程数量等于通过 i_threads 设置的线程数目。通过以

下操作大大减少了编码时间，提升了编码效率。对系统进行 pthread 多线程优化的主要操作如下：

1. 从网络上下载 win64 环境下 pthread 相关的库文件和头文件。
2. 将 pthreadVC2.lib 库的位置加入 vc++目录下子条目引用目录中，并将引用到的头文件加入包含目录。
3. 项目上点击右键，选择 c/c++项目的子条目预处理器，在预处理定义中加入 HAVE_PTHREAD 和 SYS_MINGW 两个参数。
4. 在项目命令参数中加入--threads 线程数目，或者直接修改 x264 编码参数中的 i_threads，设定为要使用的线程数量。

经过上述步骤后，就可以使用多线程进行编码操作。在本文所涉及的系统中通过配置 threads 参数以及配置进行多线程并行分块编码。以下对本论文所使用的编码程序进行了并行优化效果测试，测试序列为 FourPeople_1280x720_60.yuv，设置参数为 QP 为 25，ref 为 3，采用半像素精度进行动态搜索，并对每个块再基于 1/4 像素精度使用快速模式搜索。对使用 1、2、4 和 8 个线程进行测试，测试结果如表格 3.6 所示:

表3.6　并行测试结果

线程数	1	2	4	8
码率（kb/s）	756.86	758.51	765.26	774.28
编码速度（fps）	16.61	27.44	31.23	31.60

通过表格中的数据可以看出在本文使用的硬件平台上，当使用 4 个线程进行并行 操作时，帧率相对与不进行并行处理提升了 188%。由于使用的 CPU 是双核四线程，在使用 4 线程时帧率提升幅度最大，充分利用了多线程的优势。用这种方法编码性能的提升容易受硬件条件的影响，与 CPU 的核心数和逻辑核心数有关。本系统使用双
核四线程的 i3，使用 4 线程同时进行编码时，实现了系统性能的优化。

参考

基于x264的实时流媒体传输系统设计与实现_常良玉.caj
作者：yang_4e3c链接：https://www.jianshu.com/p/d5d92421186a来源：简书著作权归作者所有。商业转载请联系作者获得授权，非商业转载请注明出处。